

Optimal VWAP Execution: A Synthesis of Stochastic Control, Forecasting, and Adaptive Trading

Nitin Singh December 2025

sin@nitins.dev | sin.nitins@gmail.com | linkedin.com/in/sinnitin

Abstract

The Volume Weighted Average Price (VWAP) execution strategy serves as a fundamental benchmark and risk-aware tactical tool in institutional trading. Its design integrates continuous-time stochastic control, microstructure-aware volume forecasting, and adaptive optimization within a low-latency, closed-loop architecture. This paper synthesizes contemporary advances in VWAP systems, examining mathematical formulations—from the Almgren-Chriss framework and CARA utility maximization to dynamic programming and reinforcement learning—alongside volume prediction techniques ranging from component-based models (CMEM, SPAR) to machine learning and transformer-based approaches. Emphasis is placed on the coherent coupling of forecasting, scheduling, and execution layers, with practical discussion on implementation in live production environments under stochastic liquidity and evolving market regimes.

Keywords:

Core Execution Strategy:

VWAP, Optimal Execution, Algorithmic Trading, Implementation Shortfall

Mathematical & Control Frameworks:

Almgren-Chriss Model, CARA Utility, Stochastic Optimal Control, Hamilton-Jacobi-Bellman (HJB), Dynamic Programming, Reinforcement Learning (RL), Hierarchical Reinforcement Learning (HRL)

Volume Forecasting Methods:

Volume Forecasting, CMEM, SPAR, ARMA, Lasso, Ridge, XGBoost, Transformer, LSTM, Deep Learning, Machine Learning

Market & Microstructure Context:

Market Microstructure, Market Impact, Liquidity Forecasting, Intraday Volume, U-Shaped Volume Pattern

System Implementation & Adaptation:

Adaptive Execution, Closed-Loop Execution, Real-Time Systems, Low-Latency, Regime Switching, Transaction Cost Analysis (TCA)

Volume Weighted Average Price Strategy (VWAP)

Volume Weighted Average Price (**VWAP**) is primarily defined as a benchmark and strategic tool designed to minimize market impact by slicing large orders over a specific time horizon in proportion to the market's expected (forecasted) trading volume. This form of execution helps conceal the trader's intentions, aligns execution with liquidity patterns and thus leads to reduced **transaction costs**. The precision of the **volume forecasts** during VWAP execution directly translates into the algorithm's ability to track the benchmark, minimize market footprint, and ultimately reduce execution costs. VWAP is used as an **execution quality benchmark** to provide a neutral and fair measure of execution performance.



Figure 1: Core Concerns of Optimal VWAP Execution

An efficient VWAP execution setup is typically a delicate balance of these concerns and is fundamentally a **dynamic optimization problem** shaped by stochastic market evolution, microstructure frictions, and the need for adaptive, risk-aware control. Effective volume forecasting combined with precise controls leads to optimal execution of a large order by achieving the best outcomes (i.e., maximizing P&L or minimizing deviation), benchmarked against the VWAP.

The success of a VWAP strategy hinges fundamentally on accurate volume ratio predictions, allowing traders to align execution with the pronounced **"U-shaped" pattern** of market liquidity, where activity peaks during the opening and closing hours (Kim et al., 2023, p. 2; Lee & Park, 2023, p. 2). By precisely forecasting these intraday fluctuations, brokers can optimally split large parent orders into child orders to minimize transaction costs and slippage

(Cucuringu et al., 2025, p. 1; Tan et al., 2025, p. 2). To navigate complex market dynamics, modern practitioners employ sophisticated optimization control frameworks, such as **Hierarchical Reinforcement Learning (HRL)**, which automate the decision-making process across multiple temporal scales to ensure superior execution (Li et al., 2022, p. 1; Summary Text, p. 1540). Furthermore, these intelligent systems utilize a continuous **feedback loop** to adaptively adjust execution trajectories in real-time, responding dynamically to evolving market conditions and sudden volatility (Genet, 2025, p. 1; Li et al., 2022, p. 6).

Realistic VWAP Execution: A Closed Loop Process

Modern VWAP execution systems are typically implemented as a **Closed-Loop Pipeline** that integrates volume forecasting, optimization, execution and feedback. Li et al. (2022, pp. 1, 10–11), in their study of VWAP optimization using deep **Reinforcement Learning (RL)** proposed a **Macro-Meta-Micro Trader (M3T)** architecture, which captures market patterns across different temporal scales. The system relies on a workflow where the **Meta Trader** and **Micro Trader** optimize their policies according to feedback rewards received from the environment, specifically feeding back execution results to allow the system to select new subgoals based on instant liquidity.

The following setup is applicable to a typical VWAP volume forecasting and optimization pipeline:

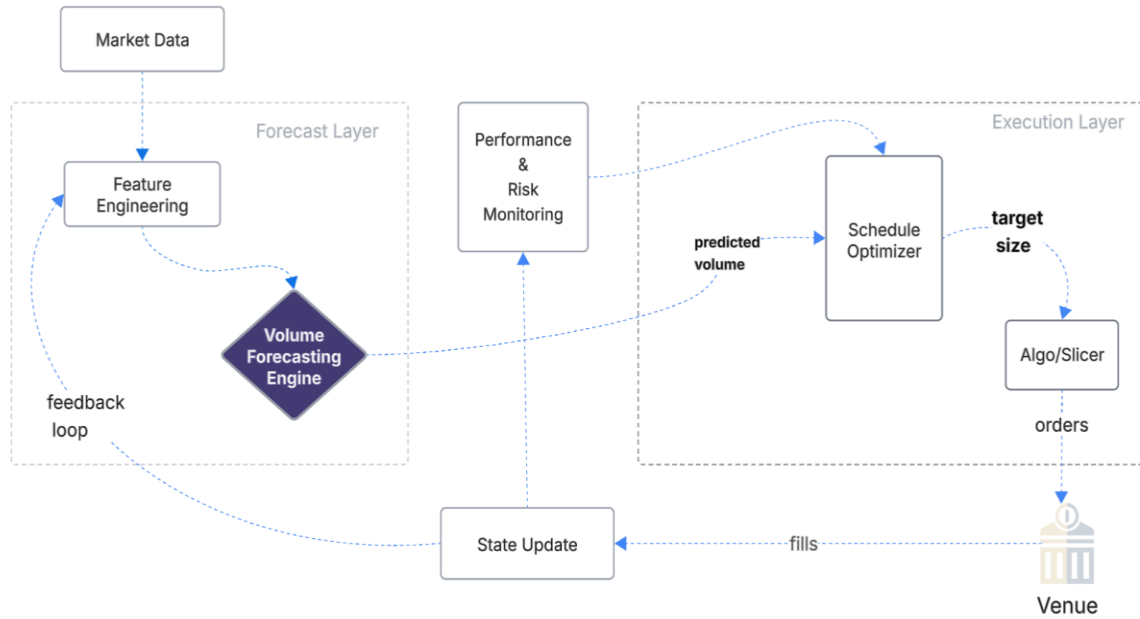


Figure 2: Typical VWAP Execution Setup

VWAP Volume Forecast Layer

The **Market Data Feeds** define the state dynamics and underlying stochastic processes governing mid-price evolution, liquidity supply, and market-impact coefficients by capturing trades, quotes, depth, and exchange events. These raw signals are transformed in the **Preprocessing & Feature Store**, where minute-level bins, relative-volume curves, and cross-sectional microstructure features construct the state vector used by both forecasting and optimization layers. This preprocessing step **discretizes** the continuous-time signals into minute-level state variables—relative volume, volatility, and cross-sectional predictors - effectively transforming the continuous-time execution problem into a tractable **Markovian representation** suitable for **Dynamic Programming (DP)** or **Reinforcement-Learning (RL)** formulations.

VWAP Execution Layer

At the execution layer, the **Almgren-Chriss (A-C) model** and its extensions provide a theoretical framework for calculating the profit and loss (**Execution Cash Process**) and incorporating risk aversion (**CARA utility maximization**) into the optimization process. In a common VWAP execution setup, volume profiles generated by the volume forecast layer directly inform the allocation strategies of large parent orders and thus connecting the forecast layer (CMEM, SPAR, statistical, ML, deep learning) with the execution layer (CARA, cash process, static/dynamic/adaptive schedules) enabling dynamic adaptation to the changing market conditions.

Schedule Optimization

The **Volume Forecast Engine** provides the predictive component of the control system, generating interval-ahead liquidity forecasts that quantify both expected market volume and its uncertainty. In A-C-style models, this corresponds to estimating the drift and volatility of the volume process; in RL settings, it defines the transition structure of the environment. These forecasts feed directly into the **Schedule Optimizer**, which solves the risk-sensitive control problem every Δt . Here, CARA-based objectives or mean–variance criteria translate into a dynamic policy that balances expected slippage against liquidation risk, adjusting participation rates in response to updated forecasts, inventory levels, and market-impact constraints.

Execution Engine

The execution engine implements the tactical layer of the policy, converting the optimizer’s slice into executable child orders. This includes limit-order placement, market-order triggers, venue routing, and microstructure-aware slicing—effectively the actuator of the control system. The resulting fill & trade data closes the feedback loop: realized fills update the state, recalibrate the volume forecast, and trigger re-optimization of the execution trajectory. This feedback mechanism is essential for stabilizing the control process under stochastic liquidity and price impact.

Performance & Risk Monitoring Layer

Finally, this layer evaluates the realized control outcomes, tracking slippage relative to VWAP, participation deviations, and risk-limit adherence. This layer corresponds to the measurement of the terminal revenue $\mathbf{X}_T$ and the empirical estimation of the utility or reward function. Its outputs feed back into model calibration, policy evaluation, and long-horizon improvements to both the forecasting and optimization layers.

These components instantiate a fully integrated execution architecture in which continuous-time stochastic control, microstructure-aware modelling, and adaptive learning jointly determine how the system decomposes X_0 into optimal child orders throughout the trading horizon.

This is precisely the structure envisioned in modern extensions of the A-C framework and RL-based execution research as noted by Białkowski et al. (2008, pp. 3, 18) who describe VWAP execution as a loop where, at each new time interval, the broker improves their forecast of future volume by incorporating the achieved (intraday) past volume to adapt the remaining strategy.

Cheng et al. (2017, pp. 3, 8, 17) in an extension of the **Almgren-Chriss (A-C) model**, derive an optimal scheduled trading rate expressed as a state-feedback law which allows the trader to adjust the execution rate dynamically based on the current position and evolving market state. Kim et al. (2023, pp. 3, 5) characterize optimal execution as a stochastic decision-making process that automates the task of using past data to make execution decisions. Their dual-level approach uses a **reinforcement learning (RL) agent** to decide how orders are executed in each interval based on feedback from public and private state information, such as the current remaining volume.

Practically, a typical forecasting and schedule optimizer activity can be zoomed in as:

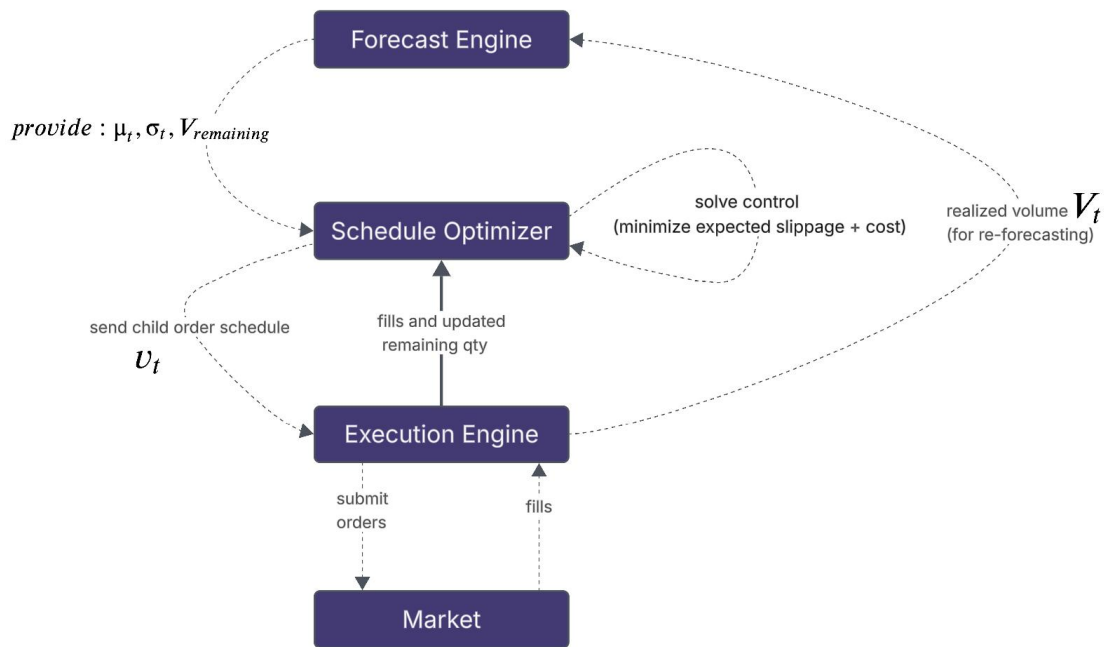


Figure 3: VWAP Schedule Optimizer Feedback Loop

1. Optimal VWAP Execution Problem: Mathematical Tools

1.1. VWAP Definition

At its core, VWAP execution can be viewed as a **continuous-time stochastic control problem** in which a trader must decompose an **initial inventory X_0** into a sequence of child orders that track the evolving market VWAP while mitigating transaction costs, market

impact, and execution risk. This perspective is consistent with Cheng et al. (2017, pp. 1–3), who emphasize stochastic control and the **Hamilton–Jacobi–Bellman (HJB)** equation as foundational tools for characterizing optimal execution policies under uncertainty in both prices and liquidity. In a similar vein, Guéant et al. (2014, pp. 1–2, 17) formulate the **guaranteed-VWAP** liquidation problem as an optimal control task, deriving an HJB partial differential equation that governs the optimal non-deterministic trading trajectory once permanent impact, temporary impact, and intraday volume profiles are incorporated into the cash-flow dynamics. From a market-microstructure standpoint, these formulations capture the interplay between order-book liquidity, impact kernels, and execution timing that determines the realized slippage relative to VWAP.

Complementing the **continuous-time approach**, Kim et al. (2023, pp. 1–4) recast the execution problem as a sequential decision-making task within a **Markov Decision Process (MDP)** framework, using reinforcement learning to automate the stochastic control of trading intensity. Their model learns a policy that adapts to real-time microstructure conditions—such as volume fluctuations, fill uncertainty, and short-horizon price dynamics—to accurately track the cumulative VWAP throughout the trading day.

The **market VWAP** over a trading horizon $[0, T]$ is defined as:

$$\text{VWAP}_T^{\text{market}} = \frac{\int_0^T S_t dQ_t}{Q_T},$$

where:

- S_t = instrument price at time t
- $Q_t = \int_0^t V_s ds$ is the cumulative market volume
- V_t (non zero) is instantaneous volume i.e., trading volume at time t
- T = total trading period (typically one day)
- n = number of time intervals

If a trader executes at speed $v_t \geq 0$ (asset units per unit time), their **execution VWAP** can be expressed as:

$$\text{VWAP}_T^{\text{exec}} = \frac{\int_0^T S_t v_t dt}{X_0}, \quad \text{where} \quad \int_0^T v_t dt = X_0.$$

The tracking error or **variance** is the difference:

$$\varepsilon_T = \text{VWAP}_T^{\text{exec}} - \text{VWAP}_T^{\text{market}}.$$

In discrete form VWAP can be expressed as,

$$\text{VWAP} = \frac{\sum_{t=1}^T S_t v_t}{\sum_{t=1}^T v_t}$$

Summation ensures that the benchmark is a "true" average of daily activity rather than a simple price average.

Example: If more units traded at \$100 than at \$101, the \$100 price has more influence on the VWAP.

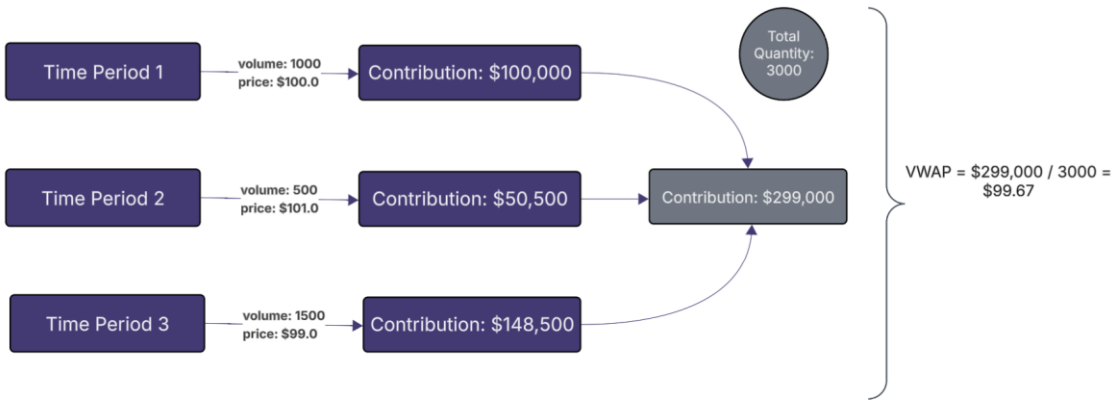


Figure 4: VWAP Calculation Example

VWAP execution is fundamentally a **dynamic optimization problem** shaped by competing forces in a stochastic and microstructure-driven environment. Each decision interval requires navigating trade-offs between:

- speed and slippage
- certainty and adaptability
- temporary cost and permanent impact, and

- the classic expected-return versus variance

The **Almgren–Chriss (A-C)** framework, the **execution cash process** X_T , and **CARA** (Constant Absolute Risk Aversion) utility provide a rigorous mathematical language for expressing these trade-offs, while modern adaptive algorithms supply the machinery to respond to real-time market conditions.

Within this setting, Barzykin and Lillo (2019, pp. 1–5, 10) reinterpret the VWAP benchmark through the perspective of the A-C model, formulating the execution cash process as the cumulative proceeds from traded quantity net of temporary-impact-driven execution costs. Their use of a CARA utility functional embeds risk preferences directly into the objective, transforming the problem into a risk-sensitive control task that accounts for the distribution of slippage relative to the VWAP benchmark. This aligns with empirical evidence across execution research showing that risk-aware criteria produce more stable performance under stochastic intraday volume and liquidity conditions.

Building on this foundation, Li et al. (2022, p. 2) adopt a **reinforcement-learning** perspective, treating the A-C structure as the underlying environment that encodes transaction-cost asymmetries, impact response functions, and risk-aversion parameters. In their formulation, VWAP execution becomes a **sequential decision process** in which the agent learns a policy that adapts participation rates to evolving market microstructure states—volume fluctuations, fill uncertainty, and short-horizon price dynamics—thereby minimizing benchmark slippage in a fully data-driven manner. Together, these approaches illustrate how continuous-time stochastic control, microstructure-aware modelling, and adaptive learning converge to shape modern VWAP execution systems.

1.2. Almgren-Chriss (A-C) Model

Formally, the **A-C model** provides the market mechanics that generate the **execution cash process** X_T , which in turn becomes the performance metric optimized under **CARA utility** to produce a **closed-form**, risk-aware trading schedule.

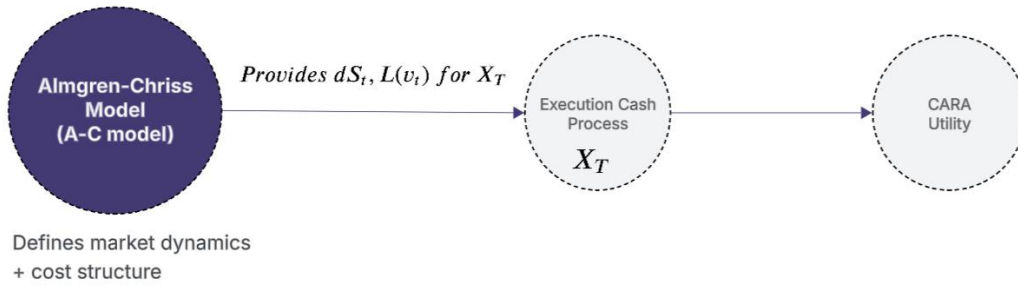


Figure 5: Role of Almgren Chriss Model in Optimal Execution

The **A-C model** assumes:

- **Price dynamics:**

$$dS_t = \sigma \cdot dW_t - k \cdot v_t dt$$

- **Temporary impact**

$$imapct_{temp}: L(v_t) = \eta \cdot v_t^2$$

Price Dynamics in A-C Model

$$dS_t = \sigma dW_t - k v_t dt$$

This stochastic differential equation (SDE) models **”how the asset’s unaffected price evolves over time”**.

Here,

- **σdW_t :** Random market fluctuations (Brownian motion).
 - σ = volatility (how “noisy” the price is).
 - W_t = standard Wiener process (captures unpredictable price moves).
- **$-k v_t dt$:** Permanent market impact $imapct_{perm}$ from trading.
 - v_t = trading rate (asset units per unit time).
 - $k > 0$ = impact coefficient: each traded asset-unit *permanently* shifts the price against the trader.
 - The minus sign means: if the trader is **selling** ($v_t > 0$), the price drifts **downward**; if buying, it drifts upward.

Measuring Permanent Impact

$imapct_{perm}$ creates a trade-off:

- Trading faster reduces **timing risk** (less exposure to random price moves).
- But it increases **permanent cost**, which directly lowers the final execution quality (since the benchmark VWAP itself may be affected if the trade is large enough).
- In guaranteed VWAP contracts, the broker must hedge this permanent cost—it's a real economic liability.

If a large institution buys 1 million units of an asset, other traders may infer that the buyer has positive private information—so they adjust their valuations upward, pushing the price higher *even after the trade is done*.

- The total effect depends on **how much a trader has traded so far**, not just the current trading speed.
- Let $X_t = \int_0^t v_s ds$ be the **cumulative executed volume** up to time t .
- Permanent impact assumes the price drifts by an amount like $k \cdot X_t$ (if linear) or more generally $f(X_t)$.

Temporary Impact in A-C Model

In real setups, the VWAP scheduler tries to **minimize temporary impact** while still executing the full order and tracking VWAP—balancing speed vs. market footprint.

$$L\left(\frac{v_t}{V_t}\right) \text{ or } L(v_t) = \eta v_t^2$$

This represents the immediate transaction cost or instantaneous cost per asset unit traded due to *temporary* market impact:

here,

- v_t : the **trading rate** at time t (e.g., units per minute).

- V_t : The **total market volume rate** at time t (how much is trading in the whole market).
- $\frac{v_t}{V_t}$: the **participation rate**—what fraction of the market’s activity trader is responsible for.
- $L(\cdot)$: A **cost function** that maps trading intensity to an **instantaneous cost per asset units** (in price units, like dollars or basis points).
- $\eta > 0$: cost coefficient (depends on liquidity, spread, order book depth).
- **Quadratic form** (v_t^2): reflects that trading faster incurs disproportionately higher costs. e.g. double the trading speed, **quadruples** the temporary cost.
- This cost is “**temporary**” because it doesn’t affect future prices - it’s the slippage trader pays *right now* to execute.

So:

- $L\left(\frac{v_t}{V_t}\right)$ means: “*The more of the market volume trader consumes, the higher their per-unit cost.*”
- Sometimes models simplify this to $L(v_t)$, assuming market volume is normalized or constant.

Thinking of liquidity as a shallow pond, dipping a cup (v_t small) barely disturbs it. Dumping a bucket (v_t large) creates a splash and leaves a temporary hole—the price trader get worsens non-linearly.

Measuring Temporary Impact

In VWAP optimization, the total expected cost includes integrating this temporary cost over time:

$$\text{Total temporary cost} = \int_0^T L\left(\frac{v_t}{V_t}\right) v_t dt$$

Essence of A-C Model (Stochastic Optimal Control with Quadratic Costs)

Putting it together, the trader’s problem becomes:

Choose a trading schedule v_t (*how fast to trade over time*) to minimize total expected cost, which includes:

- **Permanent impact cost** (via price drift in S_t),
- **Temporary execution cost** (ηv_t^2),
- **Risk penalty** from price volatility (since S_t is random).

Because **A-C model** assumes:

- Linear permanent impact (kv_t)
- Quadratic temporary cost (ηv_t^2)
- Arithmetic Brownian price noise (σdW_t)

That forms a **Linear-Quadratic-Gaussian (LQG) control problem** — one of the few stochastic control problems with **closed-form solutions** - a well-studied area with:

- Analytical or semi-analytical solutions (often involving Riccati equations or hyperbolic functions),
- Clear trade-offs between trading slowly (to reduce temporary cost) vs. trading quickly (to reduce exposure to price risk).
- allowance for computation of the optimal schedule in real-time without needing brute-force simulations.

By making these effects explicit and mathematically tractable, this framework lets traders compute **optimal execution schedules** that balance cost, impact, and risk—forming the backbone of modern algorithmic trading systems like VWAP optimizers.

1.3. Execution Cash Process (Cash received by trader up to T)

Cheng et al. (2017, pp. 4–14) extend the A-C framework by explicitly incorporating order-fill uncertainty into the liquidation problem, modelling the execution cash process as the trader's total P&L composed of the mark-to-market value of residual inventory, realized cash flows from executed shares, and a terminal block-trade penalty. Their formulation captures the **stochastic nature of execution risk** and its direct contribution to the distribution of liquidation outcomes. In parallel, Guéant and Royer (2014, pp. 1–5, 17, 20) adapt the A-C

model to the VWAP-optimization setting by embedding permanent market impact and nonlinear temporary impact into the cash-flow dynamics, defining the execution cash process as the cumulative revenue generated from selling shares under instantaneous market-volume constraints. Together, these formulations emphasize that the **execution cash process determines the realized revenue X_T** , which serves as the primary metric for evaluating execution quality in algorithmic trading.

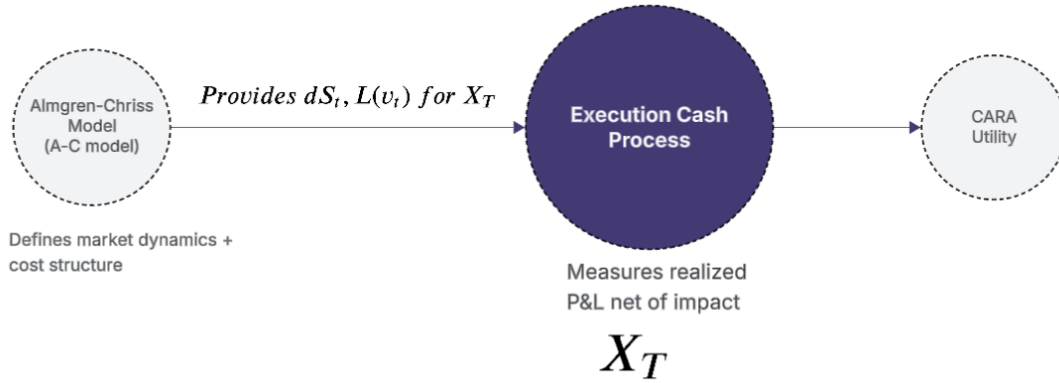


Figure 6: Role of Execution Cash Process in Optimal Execution

$$X_T = \int_0^T (S_t - \text{impact}_{temp}) \cdot v_t dt$$

This equation defines the **total cash** a trader receives from selling asset unit (or pays when buying) over the time interval $[0, T]$, after accounting for temporary market impact.

here,

- X_T : total cash accumulated by time T .
- S_t : the unaffected market price at time t — i.e., the price that would exist if the trader weren't executing orders.
- v_t : the trader's execution rate at time t (asset units per unit time).
- $\text{impact}_{temp}(v_t, V_t)$: the temporary price slippage caused by trading at rate v_t in a market with volume rate V_t .
- $(S_t - \text{impact}_{temp})$ is the actual execution price per unit - lower than S_t when selling (because the order pushes the price down temporarily).

At each instant, trader sells $v_t dt$ units at a price slightly worse than S_t , and integrates that over time to get total cash.

Subtracting the *immediate slippage* from each trade allows calculation of the **net cash inflow**, and that is what is compared against the benchmark.

Why Subtract Temporary Impact? - Temporary impact reflects the immediate cost of “**eating into**” the order book — e.g., crossing the spread or depleting liquidity at the best bid/ask. It’s called temporary because the price typically reverts after the trade (unlike permanent impact, which shifts the long-term price trend).

Thus, we don’t get the full S_t per asset unit - we get slightly less (when selling), and this formula captures that loss.

The more aggressively a trader trades relative to available liquidity (V_t), the worse is the fill price. This equation sums up all the cash trader actually collects, net of those short-term price deteriorations.

As noted X_T is central to **slippage calculation**:

$$\text{Slippage} = X_T - q_0 \cdot \text{VWAP}_T$$

(where q_0 is total units traded). The goal of optimal VWAP execution is to **maximize** X_T (or minimize negative slippage).

Remember without A-C’s assumptions, X_T is just an accounting formula. With A-C, it becomes a stochastic outcome shaped by v_t control.

Example (discrete approximation)

Suppose over 1 minute:

- $S_t = \$100$
- Trader sells 1,000 units ($v_t = 1000/\text{min}$)
- Market volume $V_t = 10,000$ units/min
- Temporary impact model: $\text{impact}_{temp} = 0.01 \cdot \frac{v_t}{V_t} \cdot S_t = 0.01 \cdot 0.1 \cdot 100 = \0.10

Then the execution price $\approx \$99.90$, and cash received $\approx \$99,900$.

For an effective VWAP schedule, minimizing *average* slippage isn't enough; low variance and penalizing uncertainty (risk aversion) are also desired attributes, which is where CARA utility maximization plays a significant role.

1.4. Risk-Aware VWAP Schedule Optimization: CARA (Constant Absolute Risk Aversion) Utility & Mean-Variance

Risk-aware optimization is the process of incorporating risk into the optimization problem. Smart execution algos don't just minimize average cost — they account for risk too.

*The intuition behind **CARA utility** is that the trader is not just maximizing cash — they're maximizing utility of cash relative to benchmark, penalizing uncertainty.*

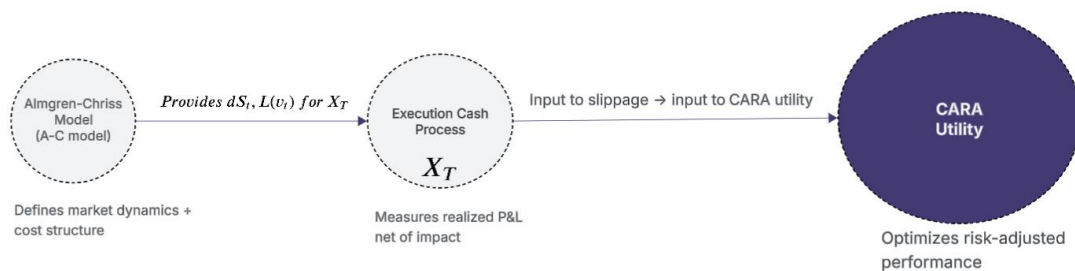


Figure 7: Role of CARA Utility in Optimal Execution

The application of the A-C framework by Guéant and Royer (2014, pp. 1–5, 17, 20) demonstrates how indifference pricing within a CARA utility setting can be used to derive both the optimal execution strategy and the premium a broker must charge to underwrite a guaranteed VWAP contract. Consistent with this perspective, Cheng et al. (2017, pp. 4–14) extend the A-C model to incorporate order-fill uncertainty during liquidation, showing that optimal policies arise from maximizing the **expected exponential (CARA) utility** of the terminal P&L. In the strict-liquidation limit, their formulation converges to an adaptive VWAP-style strategy, highlighting the structural link between risk-sensitive utility maximization and benchmark-driven execution.

Implicitly, the risk aversion in VWAP execution contexts can be specified via:

- Max participation rate (e.g., “never exceed 30% of volume”)
- Urgency (e.g., “finish by 3:30 PM no matter what”)

- Volatility scaling (e.g., “slow down if volatility > 2x average”)

Mathematically, CARA utility is defined as,

$$\max_{\boldsymbol{v}} \mathbb{E}[-\exp(-\gamma(X_T - q_0 \cdot \text{VWAP}_T))]$$

or

$$\max_{\boldsymbol{v}} \mathbb{E}[-\exp(-\gamma \cdot \text{Slippage})]$$

here,

- $\boldsymbol{v}$: The *entire trading schedule*—i.e., the choice of how fast to trade at every moment from time 0 to T . This is the control variable we’re optimizing.
- X_T : Total cash received from trading q_0 asset units by time T , after accounting for market impact (as defined in the earlier “execution cash process”).
- $q_0 \cdot \text{VWAP}_T$: The *benchmark value*—what the order would be worth if executed perfectly at the market’s VWAP.
- $X_T - q_0 \cdot \text{VWAP}_T$: This is the slippage (or *implementation shortfall*).
 - If positive $\rightarrow$ trader did better than VWAP (good when selling).
 - If negative $\rightarrow$ trader underperformed VWAP (bad).
- $\gamma > 0$: Coefficient of **absolute risk aversion**. Larger γ = more risk-averse.
- $-\exp(-\gamma \cdot \text{Slippage})$: This is the **CARA utility function** (Constant Absolute Risk Aversion), commonly used in finance because it penalizes losses more than it rewards gains and has nice analytical properties.
- $\mathbb{E}[\cdot]$: Expectation over all possible market paths (e.g., price volatility, volume randomness).

Key Observations:

- A naive strategy might minimize $\mathbb{E}[q_0 \cdot \text{VWAP}_T - X_T]$ (i.e., average cost).
- But this ignores risk: sometimes traders might get very bad outcomes due to volatility or volume surprises.
- The CARA utility penalizes variability in slippage. Even if two strategies have the same average slippage, the one with higher variance will have lower expected utility.

That effectively balances **expected performance** against **uncertainty**—exactly what a prudent institutional broker should do.

Connection to Mean–Variance

In the baseline A-C formulation, the objective is to minimize expected execution cost, but in a stochastic market environment this expectation-only criterion leaves the **Execution Layer** vulnerable to adverse price paths and liquidity shocks. Introducing CARA preferences transforms the problem into a risk-sensitive optimization, allowing the execution policy to internalize uncertainty in both price dynamics and market impact. This corresponds to shifting the **Schedule Optimizer** (refer to Figure 2) from a deterministic cost-minimizer to a controller that explicitly balances expected slippage against liquidation risk, using inputs from the **Forecast Layer** to modulate participation rates in real time.

This risk-aware perspective is echoed in the GKAC framework, where Skachkov (2013, pp. 3–5, 15) applies calculus of variations to derive optimal liquidation trajectories under a mean–variance utility functional. In system terms, the expected-return component maps naturally to the execution cash process generated by the algorithm/slicer, while the variance penalty aligns with the role of the **Performance & Risk Monitoring** module, which quantifies uncertainty in realized revenue and feeds this information back into the control loop. The resulting trajectory effectively becomes a VWAP-style schedule that is continuously adjusted based on forecasted volume, market-impact conditions, and real-time state updates.

Mathematically, under the assumption that slippage is **normally distributed** (Gaussian), maximizing this CARA utility is equivalent to solving:

$$\min_v \underbrace{\mathbb{E}[\text{slippage}]}_{\text{expected cost}} + \frac{\gamma}{2} \underbrace{\text{Var}(\text{slippage})}_{\text{risk penalty}}$$

Therefore, minimizing average slippage **plus** applying a penalty proportional to its variance (scaled by risk aversion γ).

Implications:

Old A-C model (Cost Min)	New A-C model (CARA Utility Max)
Minimize avg. slippage	Minimize avg. slippage + risk penalty

Old A-C model (Cost Min)	New A-C model (CARA Utility Max)
Ignores variance	Penalizes variance via γ
Schedule = fixed curve	Schedule = depends on γ, σ

This means:

- Optimal schedule can be computed **in real-time** — no simulations or waiting is required.
- The solution explicitly shows how γ (**risk aversion**), σ (**volatility**), k (**permanent impact**), and η (**temporary cost**) trade off against each other.
- The approach provides **analytical sensitivity** — e.g., “If volatility doubles, front-load 15% more.”

Why This Matters in VWAP Execution?

Because market conditions are uncertain: future prices (S_t) and volumes (V_t) are random and a purely deterministic schedule (e.g., static VWAP) might look good on average but fail badly on volatile days, the **CARA objective** leads to adaptive, robust strategies that:

- Trade more aggressively when liquidity is high (lower impact),
- Slow down when markets are thin or volatile (to avoid large slippage spikes).

This form of variation in trading activity is heavily dependent on the precision of **intra-day market volume forecasting**. Accurate volume forecasting is the most critical input for any successful VWAP execution strategy. Because a VWAP algorithm’s primary objective is to participate in the market in proportion to its activity, a reliable prediction of the day’s volume curve directly informs the ideal trading schedule.

Volume forecasting methods, such as the Component Multiplicative Error Model (**CMEM**) and the Separated Periodic Autoregressive (**SPAR**) model, are integral parts of the optimal execution problem, providing the essential input for sophisticated frameworks like the Almgren-Chriss (AC) model and deep learning architectures that optimize the resulting execution cash process (Profit and Loss or P&L). Such techniques predict the distribution of trading activity, which is crucial for determining the optimal schedule of child orders.

Emphasizing the significance of volume forecasting, Białkowski et al. (2008, pp. 1–3, 17–18) argue that the only variable required to track the VWAP benchmark is the **intraday volume**

shape, as predicting the price itself is not necessary for successful replication. They propose a methodology that decomposes volume into a common market component and a stock-specific component, using **Principal Component Analysis (PCA)** to remove seasonal variations without imposing a fixed parametric form. Their research demonstrates that identifying these components allows for dynamically updated predictions that significantly lower the risk of failing to meet the benchmark.

In their study of U-shaped intra-day volume pattern, Kim et al. (2023, pp. 1, 3, 5–6) leverage the pattern to distribute orders accurately over a full-day horizon thus proposing a dual-level architecture where a "**U-shape Transformer**" model is specifically designed to capture day-to-day variations in global daily volume distributions. They suggest that capturing these global volume oscillations is essential because variations in financial data are often insignificant within shorter trading windows, making standard forecasting difficult.

Characteristics of U-shaped Intra-day Volume Pattern:

1. **High Opening Volume:** Information accumulated overnight creates trading pressure
2. **Mid-Day Lull:** e.g., reduced activity during 10:00 AM - 2:30 PM
3. **Closing Surge:** Portfolio rebalancing and auction participation

The intraday volume can be decomposed as:

$$x_{t,j} = \eta_t \cdot S_j \cdot \mu_{t,j} \cdot \epsilon_{t,j}$$

observed volume

error term (random noise)

intra day dynamic component (auto correlation effects)

intra day periodic component (U-shaped pattern)

average daily volume (overall market activity)

Figure 8: Intra Day Volume U-Shaped Pattern

The interesting research performed by Mitchell et al. (2013, pp. 1–4, 15–17) provides an optimal dynamic solution that depends on the correlation between stock returns and trading volume, as well as the expected volume itself. They develop a **cross-sectional** model of volume that incorporates **shape factors**, **day-of-the-week effects**, and **autocorrelation** (intraday and overnight). They note that intervals with high expected volume have a disproportionate impact on VWAP calculation, necessitating highly accurate forecasting in those specific periods.

2. VWAP Volume Forecasting Methods

Accurate volume forecasting is critical for VWAP execution and various approaches fall into volume decomposition schemes, statistical and machine learning methods and more recently deep learning techniques.

2.1 Volume Decomposition Approaches for Forecasting

Strategies have been developed to explicitly model and separate the deterministic periodic component of volume from the random fluctuations. **Component Multiplicative Error Model (CMEM)** and **Separated Periodic Autoregressive (SPAR)** models are two such popular approaches.

CMEM explicitly models the periodic component of volume while **SPAR** treats intraday periodicity as a time-of-day fixed effect, allowing for consistent estimation of the underlying stationary dynamics, thereby enhancing predictive accuracy.

2.1.1 Component Multiplicative Error Model (CMEM)

CMEM is studied in the volume forecasting research performed by Cucuringu et al. (2025, pp. 268, 275–276, 290–291, 326). They utilize CMEM as a primary decomposition framework, which breaks volume into three components:

- a **daily component** (day-to-day trends),
- an **intraday periodic component** (time-of-day patterns),
- and an **intraday non-periodic component** (autocorrelated dynamics).

The study also references **STL decomposition**, **wavelet decomposition**, and the **Additive Error Model**, which uses simultaneous cross-sectional and time-series data to articulate the intraday U-shape. Feature importance analysis in this study confirms that these decomposition components are the most influential predictors for machine learning-based forecasting models.

CMEM is a **multiplicative error model** that explicitly models the **periodic** component of volume.

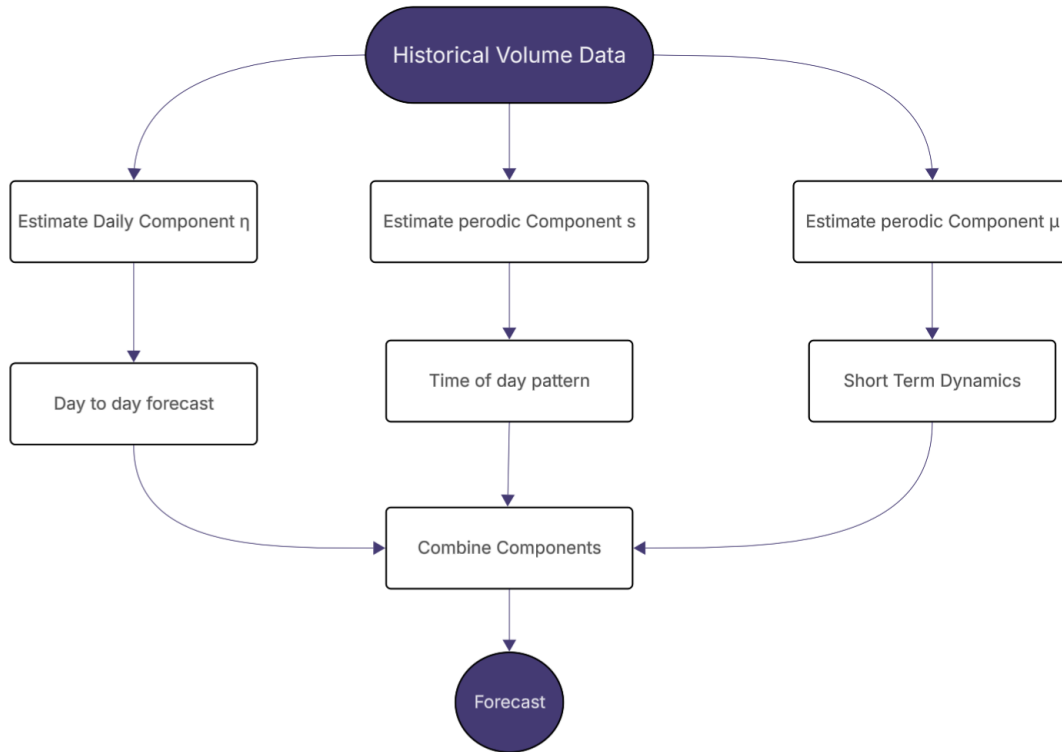


Figure 9: CMEM Volume Decomposition Workflow

$$x_{t,j} = \eta_t \cdot s_j \cdot \mu_{t,j} \cdot \epsilon_{t,j}$$

Component Details:

1. Daily Component (η_t): η_t captures day-to-day persistence

Example: If Monday had high volume ($\eta_{monday} = 1.2$), Tuesday might also see elevated volume ($\eta_{tuesday} = 1.15$)

Follows autoregressive structure:

$$\eta_t = \omega + \alpha \cdot \eta_{t-1} + \beta \cdot \varepsilon_{t-1}$$

2. Intraday Periodic Component (s_j): s_j represents the U-shape pattern

Example time-of-day multipliers: 9:30-10:00 AM: $s_1 = 1.8$ (80% above average) 12:00-12:30 PM: $s_{15} = 0.6$ (40% below average) 3:30-4:00 PM: $s_{26} = 2.2$ (120% above average)

3. Intraday Dynamic Component ($\mu_{t,j}$): $\mu_{t,j}$ captures short-term autocorrelation

If volume in previous interval was high, current interval likely also elevated:

$$\mu_{t,j} = \omega_\mu + \alpha_\mu \cdot (x_{t,j-1}/(\eta_t \cdot s_{j-1})) + \beta_\mu \cdot \mu_{t,j-1}$$

CMEM Forecasting Process:

Forecast: $\hat{x}_{t,j} = \hat{\eta}_t \times \hat{s}_j \times \hat{\mu}_{t,j}$

CMEM captures the fundamental volume dynamics, allows interpretable components and is computationally efficient. But is statistically Univariate (ignores cross-stock information), requires positivity constraints and has limited predictive power ($R^2 \approx 0.265$).

2.1.2 Separated Periodic Autoregressive (SPAR) Model

SPAR is another volume decomposition method designed for forecasting intraday trading volume by addressing the critical challenge of pronounced periodicity in financial markets.

SPAR model is the focus of research by Tan et al. (2025, pp. 117, 121, 125, 137–138, 145) which treats intraday periodicity as a time-of-day fixed effect. The method uses a **within-transformation technique** to remove the deterministic periodic component, resulting in a stationary series that can be modelled via standard autoregressive frameworks.

This approach contrasts with the CMEM model, which requires the functional form of the periodic component (like splines) to be specified *ex ante*. The within-transformation is specifically designed to isolate predictable patterns from the more random daily fluctuations in trading volume. By treating intraday periodicity as a time-of-day fixed effect, the model isolates predictable patterns from random daily fluctuations. Applying these forecasts to a dynamic replication strategy substantially reduces tracking error and lowers transaction costs.

The core principle of the SPAR is to explicitly separate the volume data into two distinct components to simplify the forecasting task:

1. **A deterministic periodic component:** This captures the predictable, systematic fluctuations of trading volume that occur throughout the trading day, such as the widely known U-shaped pattern. SPAR treats this intraday periodicity as a time-of-day fixed effect.
2. **A stationary component:** This remaining component represents the unpredictable, dynamic fluctuations in volume after the deterministic periodicity has been removed.

SPAR extends decomposition methods by explicitly isolating the deterministic periodic component of volume, treating it as a time-of-day fixed effect that is removed through within-transformation. This separation allows for more accurate forecasting of the underlying stationary dynamics. Empirically, applying SPAR forecasts to dynamic VWAP replication strategies reduces **tracking error** and enhances algorithmic execution efficiency, demonstrating the economic significance of precise volume decomposition.

How Decomposition Works in SPAR?

The method achieves separation through a technique called **within-transformation** (or demeaning), which eliminates the periodic component from the data prior to estimation. This removal allows for consistent and efficient parameter estimation of the underlying stationary dynamics using a standard autoregressive framework.

Compared to CMEM

This decomposition approach relates to established methods like the **CMEM**, which also decomposes volume into periodic and non-periodic components. While CMEM often requires specifying the functional form of the periodic component (e.g., a U-shape modeled by splines), SPAR treats the periodic component **nonparametrically** as a fixed effect, which is removed using the robust within-transformation method. This difference helps SPAR avoid the risk of model misspecification when the actual periodic shape deviates from a chosen parametric form, such as J-shaped or W-shaped curves observed in various markets.

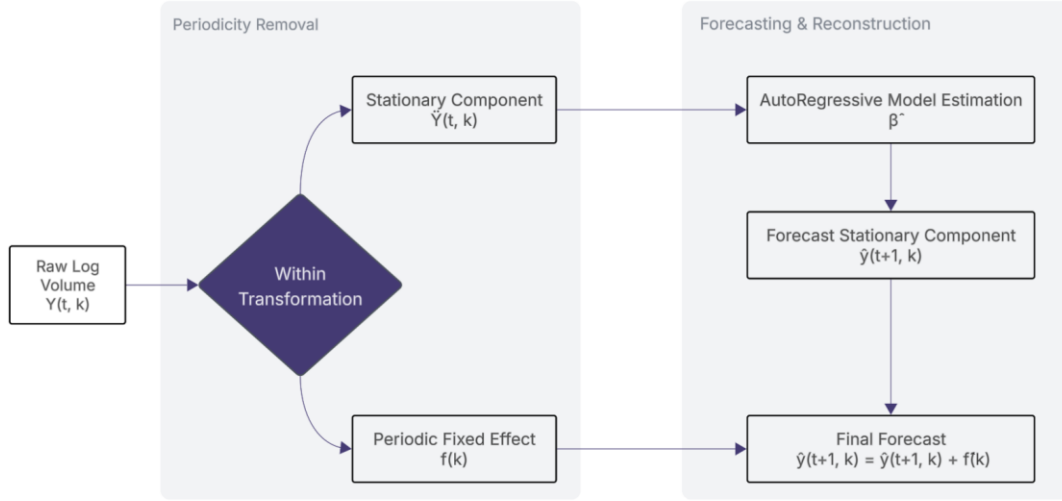


Figure 10: SPAR Volume Decomposition Workflow

SPAR Methodology

1. **Treating Periodicity as a Fixed Effect:** Instead of attempting to model the complex, specific shape of the intraday volume curve (like the known U-shaped pattern), the SPAR model treats the intraday periodicity as a time-of-day fixed effect. This re-conceptualization allows for a unified and robust approach to handling different intraday patterns observed across various assets, such as U-shaped or J-shaped curves.
2. **Periodicity Separation via Within-Transformation:** The crucial mathematical step is the **within-transformation** (or demeaning). This standard panel data technique eliminates the periodic component before estimation. This process involves calculating the cross-day mean for each time-of-day interval and then subtracting this mean from the raw data to isolate the non-periodic, stationary series.
 - For a process $Z_{t,k}$ (e.g., log trading volume) indexed by day t and time interval k , the within-transformation yields the demeaned series $\ddot{Z}_{t,k} = Z_{t,k} - \bar{Z}_{t,k}$, where $\bar{Z}_{t,k}$ is the cross-day mean over a moving window W .
 - Since the periodic fixed effect (f_k) does not vary by day, applying the within-transformation removes it ($\ddot{f}_k = 0$), allowing the model to consistently estimate the remaining underlying stationary dynamics.
3. **Modeling the Stationary Component (Autoregressive):** Once the periodic component is removed, the remaining stationary series ($\ddot{y}_{t,k}$) can be effectively

modeled using a standard **autoregressive framework**. This is typically formulated as a panel data regression:

$$\dot{y}_{t,k} = \ddot{X}_{t,k-1}\beta + \ddot{u}_{t,k}$$

where $\ddot{X}$ represents the demeaned predictors (features) and β is the vector of coefficients estimated, usually via Ordinary Least Squares (OLS) on the transformed data.

4. **Forecast Construction and Interpretation:** The final one-step-ahead forecast ($\hat{y}_{t+1}$) is then constructed by combining the model's output on the demeaned components with the estimated periodic component, recovered by taking the average of the observed data ($\bar{y}_{t,k}$) and adjusting for the mean of the features multiplied by the estimated coefficient $\hat{\beta}$. This process ensures the forecast incorporates both the time-of-day fixed effect and the recent dynamics.

The SPAR framework offers significant practical and analytical benefits:

- **Model-Agnostic Preprocessing:** SPAR acts as a data preprocessing technique that can be integrated with various forecasting methods, including OLS, Lasso, Ridge, and Random Forest, consistently enhancing predictive accuracy across these models compared to their unadjusted counterparts.
- **Enhanced Predictive Accuracy:** Empirical results show that SPAR models significantly outperform standard benchmarks, including the sophisticated **Kalman Filter (KF)** model and non-regularized OLS models, because the method explicitly accounts for intraday periodicity.
- **Structural Analysis:** SPAR provides an explicit, interpretable estimate of the **intraday periodic curve** conditional on explanatory variables. Factor analysis conducted on these estimated curves revealed that volume patterns are fundamentally driven by two common factors: a U-shaped curve and a morning-versus-afternoon asymmetry.
- **Economic Significance:** Applying SPAR forecasts to a dynamic VWAP replication strategy demonstrated substantial economic value by reducing tracking error and enhancing algorithmic execution efficiency.

The SPAR model effectively separates the complex, deterministic historical patterns from the random, predictable fluctuations, resulting in a cleaner signal for time-series forecasting. The central mechanism, **the within-transformation**, removes $f(k)$ to model only the stationary portion D , ensuring the predictive model E estimates dynamics untainted by the fixed, daily seasonal pattern. The removed periodic component C is then added back to construct the complete forecast G .

The core idea of the SPAR model is to simplify the forecasting task by explicitly separating the volume data into two distinct components: a **deterministic periodic component** and a **stationary component**.

SPAR decomposition is leveraged as a preprocessing step that enhances predictive accuracy across various statistical and machine learning algorithms.

2.2 Statistical Approaches for Volume Forecasting

Referring back to **Figure 2**, in the **Preprocessing & Feature Store**, the earliest approaches—such as the **Rolling Mean**—provide a naive yet surprisingly robust baseline, capturing the stable intraday seasonality that dominates many volume curves. Classical time-series models like **ARMA** introduce short-horizon dynamics while remaining analytically tractable, and **CMEM** extends this lineage by combining deterministic intraday profiles with multiplicative stochastic components grounded in economic microstructure. These models align naturally with a continuous-time view of liquidity as a stochastic process with both predictable and random components.

2.2.1. Rolling Mean (RM)

Rolling mean is a simple statistical approach, and there are no parameters to estimate, is robust to model misspecifications, it captures intraday seasonality implicitly since it's always comparing like-with-like (same clock time), it captures the U-shaped intra-day volume pattern.

- **Ignores day-to-day shifts:** if today is an earnings day or a market crash, the past 5 days won't reflect that.

- **No adaptation within the day:** can't react to morning volume surprises when forecasting afternoon buckets.

Lags structural changes: if market microstructure evolves (e.g., new ETF flows), the rolling window may be slow to adapt.

Rolling mean averages same interval over past **D** days

$$\hat{x}_{t,j} = (1/D) \sum_{d=1}^D x_{t-d,j}$$

For instance,

- By predicting volume in **interval j** (e.g., 10:00–10:15 AM) on **today (t)**.
- Instead of modeling complex dynamics, it simply takes the **average volume** observed in that *same time interval* over the **past D days**.
- e.g., If $D = 5$, it averages the 10:00–10:15 AM volume from the last 5 trading days.
- e.g., to forecast 10:00–10:15 AM volume on Friday, average the 10:00–10:15 AM volume from previous 5 days

2.2.2. ARMA Models (Autoregressive Moving Average)

ARMA offers short-term autocorrelation e.g., if volume was higher than usual at 10:00 AM, it's likely also elevated at 10:15 AM. Shocks (ε) fade over time due to mean-reversion.

ARMA is more **adaptive than Rolling Mean**— it can respond to recent deviations. But it assumes **stationarity** after de-seasonalizing (may not hold on volatile days), it doesn't model **cross-interval** or **cross-asset dynamics**, and requires careful estimation of ϕ , θ , and the seasonal baseline.

ARMA here is applied **within each interval across days** (not across intervals within a day), unless extended to a vector form.

ARMA models residuals after removing seasonal patterns

$$Residual_{t,j} = x_{t,j} - seasonal_j$$

Fit: $Residual_{t,j} = \phi \cdot Residual_{t,j-1} + \theta \cdot \varepsilon_{t,j-1} + \varepsilon_{t,j}$

It's a two-step process:

Step 1: Remove seasonal pattern

First, subtract a fixed intraday profile (like the s_j from CMEM):

$$\text{Residual}_{t,j} = x_{t,j} - \text{seasonal}_j$$

Here, seasonal_j could be the long-run average volume for interval j (e.g., from historical data).

This isolates the “**unexpected**” part of volume on day t in interval j .

Step 2: Model residuals with ARMA

Fit a time-series model to these residuals:

$$\text{Residual}_{t,j} = \phi \cdot \text{Residual}_{t,j-1} + \theta \cdot \varepsilon_{t,j-1} + \varepsilon_{t,j}$$

- **AR (Autoregressive) term:** $\phi \cdot \text{Residual}_{t,j-1}$
→ Today's deviation depends on yesterday's (or last interval's) deviation.
- **MA (Moving Average) term:** $\theta \cdot \varepsilon_{t,j-1}$
→ Accounts for short-term shock persistence (e.g., a surprise news event affecting multiple intervals).

Comparison with CMEM

Method	Captures U-shape?	Adapts to daily shifts?	Uses recent surprises?	Complexity
Rolling Mean	Implicitly	No	No	Very Low
ARMA	via pre-adjustment	Only via residuals	Yes (short-term)	Low
CMEM	Explicitly	Yes (η_t)	Yes ($\mu_{t,j}$)	Medium

Modern **ML and Transformer-based forecasters**, integrated into the **Volume Forecast Engine**, push further by incorporating nonlinearities, cross-sectional signals, and uncertainty quantification. Their ability to ingest richer microstructure features—order-book imbalance, volatility bursts, event-driven liquidity shocks—allows the system to anticipate

high-variability regimes that classical models struggle to capture. Yet, in practice, many production architectures still retain the **Rolling Mean** as a fallback model, ensuring stability when ML components fail or encounter out-of-distribution conditions.

The academic debate mirrors this architectural layering. Lee et al. (2023, p. 18) note that much of the historical literature relied on statistical models such as ARMA and GARCH, which are well-suited to the stylized statistical properties of intraday volume curves but may lack the contextual awareness needed to forecast extreme volume ratios. This limitation becomes particularly relevant for the **Schedule Optimizer**, whose control decisions depend critically on accurate forecasts of liquidity and its variance.

A contrasting view is offered by Białkowski et al. (2008, pp. 9, 25), who apply an ARMA(1,1) model to the “specific” component of intraday volume after removing the common market factor. Their findings suggest that simple time-series models can still deliver highly accurate forecasts, materially reducing the execution risk associated with tracking the VWAP benchmark. In system terms, this reinforces the value of maintaining lightweight, interpretable models within the **Volume Forecast Engine**, especially for stabilizing the control loop when microstructure conditions are benign.

These perspectives highlight that volume forecasting is not a single model but a **layered forecasting stack**:

- statistical baselines for robustness,
- econometric models for structure,
- and ML architectures for nonlinear microstructure adaptation.

This layered design ensures that the downstream **Schedule Optimizer**, **Execution Engine**, and **Performance & Risk Monitor** (refer to **Figure 2**) operate on forecasts that are both stable and responsive to the stochastic nature of real-world liquidity.

2.3 Machine Learning (ML) Approaches for Volume Forecasting

Treating VWAP optimal execution as a sequential decision-making task, the ML models automate the decision process using historical data, enabling strategies to dynamically adapt to market conditions and capture complex microstructure features. Additionally, **Deep**

Reinforcement Learning (RL) techniques have gained traction as they manage high-dimensional data and market complexity without relying on strict market assumptions.

In practice, volume decomposition methods like **Separated Periodic Autoregressive (SPAR)** are integrated with regularized statistical methods as a highly effective approach for VWAP volume forecasting.

A study performed by Tan et al. (2025, pp. 3, 11–12, 14) proposes that the decomposition based Separated Periodic Autoregressive (**SPAR**) method can be modelled via standard **autoregressive frameworks**. The authors demonstrate the "universal effectiveness" of their framework by integrating it with various models, including **Lasso and Ridge regression**. They use specific mathematical formulations for **Lasso** to promote model sparsity through coefficient shrinkage and **Ridge regression** to reduce variance caused by multicollinearity.

2.3.1. The Core Problem

Regression methods such as OLS, Lasso, Ridge minimize prediction error, but with different constraints:

Goal: Find coefficients β that minimize prediction error

Ordinary Least Squares (OLS) - The Baseline

$$\hat{\beta}_{OLS} = \operatorname{argmin}_{\beta} \sum_{i=1}^n (y_i - \mathbf{x}_i \beta)^2$$

But it is unstable when features are correlated or numerous.

The Regularization Framework

Both Lasso and Ridge add a **penalty term** to the OLS objective:

$$\hat{\beta} = \operatorname{argmin}_{\beta} \left\{ \underbrace{\sum_{i=1}^n (y_i - \mathbf{x}_i \beta)^2}_{\text{Prediction Error (L}^2 \text{ loss)}} + \underbrace{\lambda \cdot \text{Penalty}(\beta)}_{\text{Regularization Term}} \right\}$$

where:

- y = volume to be predicted
- $\mathbf{x}$ = features (historical data, autocorrelation, volatility)
- β = regression coefficients (what we're solving for)
- λ = regularization strength (higher λ = stronger penalty)

Lasso Regression (L1 Regularization)

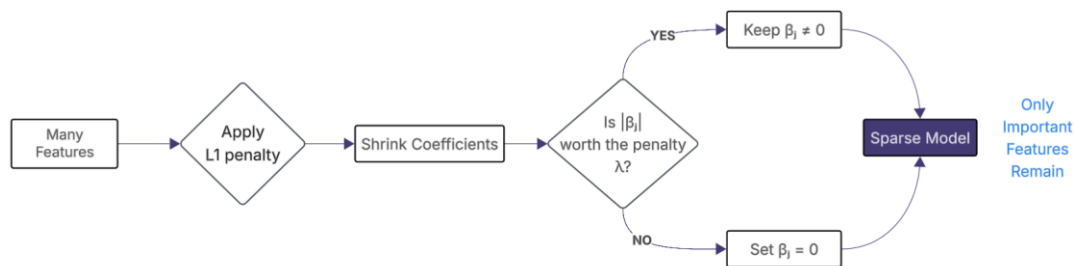


Figure 11: LASSO Regression Process

The L1 Penalty: Sum of Absolute Values

$$\hat{\beta}_{lasso} = \underset{\beta}{\operatorname{argmin}} \left\{ \sum_{i=1}^n (y_i - \mathbf{x}_i \beta)^2 + \lambda \sum_{j=1}^p |\beta_j| \right\}$$

$$\text{Penalty} = \lambda \sum_{j=1}^p |\beta_j| = \lambda (|\beta_1| + |\beta_2| + \dots + |\beta_p|)$$

Lasso Characteristic: Automatic Feature Selection

- The L1 penalty forces coefficients to exactly zero when they're not important enough to justify the penalty cost.
- The λ penalty shrinks some coefficients to exactly zero
- Performs automatic feature selection
- Prevents overfitting when many correlated predictors exist

Example: Imagine predicting volume with multiple features:

Feature	Without Lasso	With Lasso ($\lambda=0.5$)
Lag 1 volume	0.45	0.42 ✓
Lag 2 volume	0.23	0.18 ✓
Volatility	0.15	0.12 ✓
Buy notional	0.08	0.00
Sell notional	0.06	0.00
Time of day	0.04	0.00
Day of week	0.02	0.00
...	...	...

Result: 3 features instead of 10 → simpler, more interpretable model!

Ridge Regression (L2 Regularization)



Figure 12: Ridge Regression Process

The L2 Penalty: Sum of Squared Values

$$\hat{\beta}_{ridge} = \underset{\beta}{\operatorname{argmin}} \left\{ \sum_{i=1}^n (y_i - \mathbf{x}_i \beta)^2 + \lambda \sum_{j=1}^p \beta_j^2 \right\}$$

$$\text{Penalty} = \lambda \sum_{j=1}^p \beta_j^2 = \lambda(\beta_1^2 + \beta_2^2 + \dots + \beta_p^2)$$

Key Characteristic: Stability Through Shrinkage

- The L2 penalty **shrinks all coefficients toward zero** but **never forces them to exactly zero**.
- Shrinks all coefficients toward zero (but not exactly to zero)
- Handles multicollinearity well
- More stable when predictors are correlated

Example: Same features with multicollinearity:

Feature	Without Ridge	With Ridge ($\lambda=0.5$)
Lag 1 volume	0.45	0.38 ↓
Lag 2 volume	0.23	0.20 ↓
Volatility	0.15	0.13 ↓
Buy notional	0.08	0.06 ↓
Sell notional	0.06	0.05 ↓
Time of day	0.04	0.03 ↓
Day of week	0.02	0.02 ↓
...	...	...

Result: All features remain but with more stable, reliable coefficients!

2.3.2. Linear Models with Regularization - Why Regularization (Constrained Optimization) Matters?

When forecasting intraday trading volume using models like SPAR (Separated Periodic Autoregressive), we face two critical challenges:

1. Multicollinearity - features like buy notional value, sell notional value, and traded amount are highly correlated
2. High-dimensional data - many predictors (lags, volatility indicators, time features) relative to observations

How Regularization Helps in Volume Forecasting?

1. Autocorrelation Features - many lags create multicollinearity
2. Market Indicators - volatility measures often correlate with each other

3. Auxiliary Predictors - buy notional, sell notional, traded amount overlap
4. Sample Size - limited historical data relative to features

Lasso and Ridge regression add “**penalties**” to control model complexity and improve stability. e.g., use Lasso when testing 50 technical indicators to predict volume, expecting only 5-10 matter and use Ridge when using buy/sell flows, notional values, and traded amounts (all related and should all contribute).

2.3.3. Lasso and Ridge in Volume Forecasting

Tan et al. (2025, pp. 3, 11–12, 14–16, 34–35) emphasize the "universal effectiveness" of **SPAR** framework by integrating it with various linear models, specifically naming the **Lasso-SPAR (LASP)** and **Ridge-SPAR (RISP)** variants e.g.,

- **LASP** (Lasso-SPAR): applies **Lasso** regression to the **SPAR within-transformed** data, which allows for automatic feature selection and model sparsity while accounting for periodicity. It integrates L1 regularization with periodic decomposition and automatically selects relevant time periods and lags. LASP is best when there are many potential predictors, but only some are truly predictive.

- **RISP** (Ridge-SPAR): utilizes **Ridge** regression to address multicollinearity in volume-related independent variables after the periodic component has been separated. It integrates L2 regularization with periodic decomposition and stabilizes estimates when volume drivers are correlated. RISP is best when buy/sell flows, notional values, and amounts move together.

The research demonstrates that **LASP** and **RISP** consistently outperform their unadjusted counterparts (standard Lasso and Ridge models) in terms of out-of-sample and error control metrics like **RMSE** and **RMAE** across various trading volume regimes. Comprehensive asset-level results for LASP and RISP across high, medium, and low-volume stocks are detailed in the study's performance tables and appendix.

2.3.4 Non-Linear Models with Regularization - XGBoost (Gradient Boosted Trees)

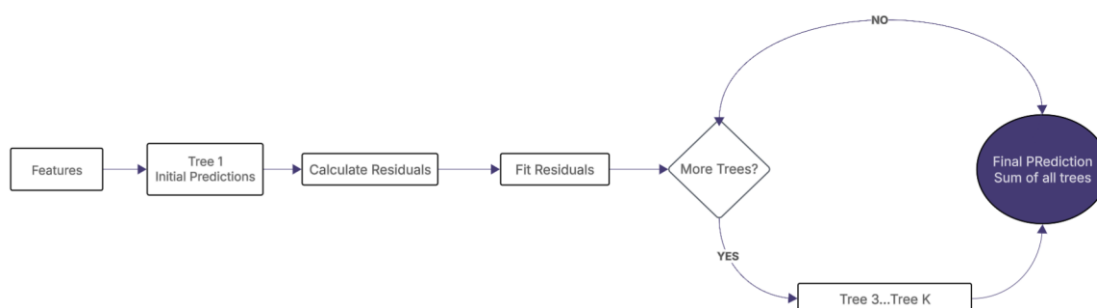


Figure 13: Regularization Process In XgBoost

Unlike **Lasso**, and **Ridge** regression, which are linear models with **built-in shrinkage**, **gradient boosting** methods such as **XGBoost** are nonlinear ensemble learners that perform implicit feature selection through their tree-building process. They differ significantly in their approach to feature selection, regularization, and model complexity. important features through tree construction.

All three are **supervised learning** methods and aim to predict future volume (target variable) given historical data (time of day, past volume, volatility, order flow, etc.) and learn patterns to forecast the next value. While Lasso/Ridge attempt to fit a straight line, but punish big coefficients, XGBoost focuses on building many decision trees, each correcting the last one's mistakes. It thrives in volume forecasting because:

- **Non-linear nature of Volume forecasting** – it depends on complex interactions e.g., volume spikes at market open (**time-of-day effect**), surges when volatility jumps and index futures roll (**interaction effect**), and decays after lunch unless news hits (**conditional pattern**). Linear models like Lasso/Ridge struggle here because they assume additive, linear relationships.
- **Captures Nonlinearities** - trees naturally model thresholds e.g., “*If volatility > 2% AND time < 10:30 AM then predict high volume*”

- **Handles Interactions Automatically** – there is no need to manually create volatility × time-of-day features, the tree finds them via splits.
- **Robust to Irrelevant Features** - if a feature (e.g., news sentiment) doesn't help reduce error, trees just won't split on it (implicit feature selection).
- **Adapts to Regime Shifts** – during market microstructure changes (e.g. new ETF flows), XGBoost can adapt if retrained unlike rigid linear assumptions.

In context of volume forecasting, decision-tree-based ensemble methods (e.g., XGBoost) are studied in research by Cucuringu et al. (2025, pp. 12–13, 17–18, 36). This study leverages gradient boosting to capture the complex, non-linear relationships and predictor interactions inherent in market dynamics. The authors utilize XGBoost across three training schemes: Single Asset (**SAM**), Clustered Asset (**CAM**), and Universal Asset (**UAM**). Their empirical results demonstrate that:

- Universal modelling significantly boosts XGBoost performance, with its out-of-sample increasing from 0.532 (SAM) to 0.622 (UAM).
- The model is used for feature importance analysis, employing a "weight" metric to quantify how frequently specific features are used to split data across the ensemble.
- The study incorporates **Order Flow Imbalance (OFI)** predictors into the XGBoost framework, finding that **absolute-magnitude OFI** features are more robust for volume prediction than directional cues, improving performance by 2.8 basis points in nonlinear settings.

2.4 Deep Learning Techniques for Volume Forecasting

Deep-learning approaches for VWAP volume forecasting are often integrated into hierarchical or specialized neural architectures. They highlight a paradigm shift toward **dynamic volume estimation**, where volume patterns are separated into broader market evolutions and specific asset class patterns to handle the dynamic nature of modern markets. Genet (2025, pp. 1–5, 8–10) proposed a framework that uses a **learnable base volume curve** as a foundation for real-time adjustments based on evolving market conditions.

2.4.1. Recurrent Neural Networks (RNNs) for Temporal Dependence

RNNs, specifically **Long Short-Term Memory (LSTM)** networks, are used to capture the temporal patterns and long-term dependencies inherent in sequential market data. This is critical because volume forecasting relies heavily on persistence and historical context.

While not purely a forecasting model, **Hierarchical Reinforcement Learning (HRL)** architectures incorporate volume forecasting (often using RNNs) as their **Macro-level** component to determine the initial allocation schedule.

Emphasizing on HRL for VWAP strategy optimization, Li et al. (2022, pp. 1–3, 8, 10) propose a **Macro-Meta-Micro Trader (M3T)** framework where the **Macro Trader** is dedicated to volume profile estimation. Using **Long Short-Term Memory (LSTM) networks**, this layer estimates future volume profiles to allocate parent orders into tranches that match market liquidity. Their analysis confirms that accurately estimating volume profiles effectively reduces transaction costs and slippage.

HRL Component	Role in Forecasting/Allocation	Technique Used
Macro Trader (Level 1)	Estimates future volume profiles and allocates the large parent order into smaller tranches (O_l) proportional to this forecast.	LSTM networks are often used as the backbone to capture temporal volume patterns.
Meta/Micro Traders (Level 2/3)	Handle dynamic adjustments to the trading rate within the small tranches based on micro-level market information (LOBs, liquidity) to minimize slippage relative to the Macro allocation target.	Deep Reinforcement Learning (e.g., DDQN, PPO) guides execution based on current market state.

Intuition (LSTM):

The core idea relies on the **Hidden State** (h_t) and **Cell State** (c_t), which propagate information through time (τ) while regulating information flow using sigmoid (σ) and tanh activation functions (e.g., forget gate f_t):

$$h_t = R(x_t, h_{t-1})$$

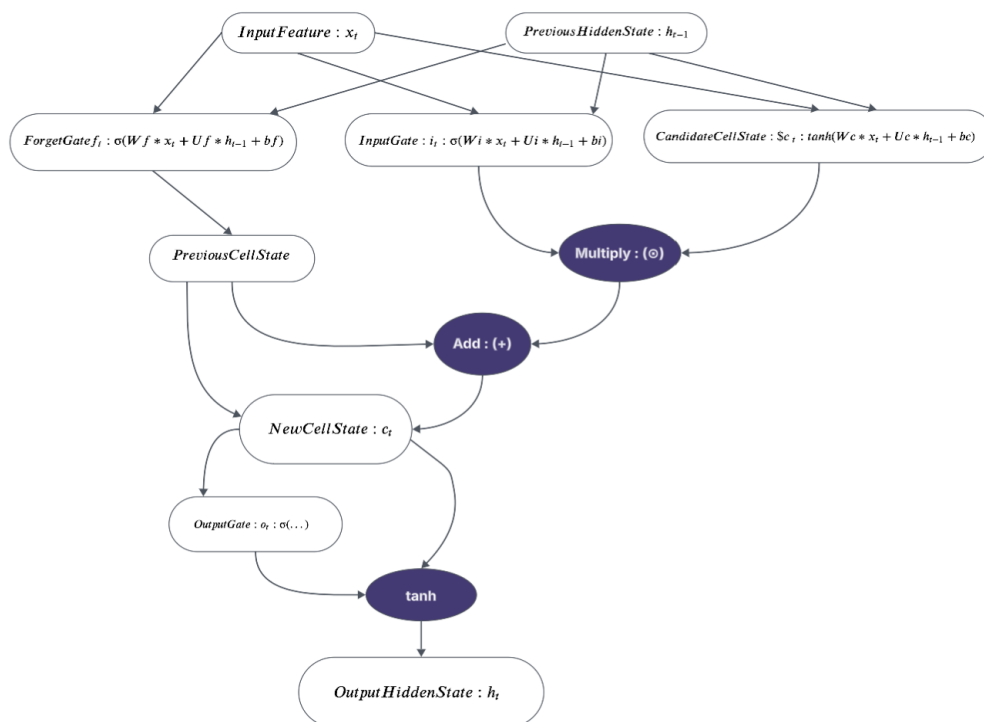


Figure 14: LSTM Workflow

This sequence allows the model to learn the time-dependent weights required to predict future volume, moving beyond simple historical averages.

Model	Technique	Intuition
LSTM-based Forecasters	Uses LSTM units with gates (forget, input, output) to selectively retain or discard information over long sequences of past volume profiles.	This approach acts like a dynamic memory system, allowing the model to remember significant past volume fluctuations across days while forgetting less

Model	Technique	Intuition
		relevant noise, making it effective for capturing overall daily patterns.
Recurrent Block (in Dynamic VWAP)	In the Dynamic VWAP execution framework, an RNN, such as a Temporal Kolmogorov-Arnold Network (TKAN) or LSTM, processes market observations sequentially to produce hidden states (h_t) that represent the cumulative market state up to time t .	The recurrent block serves as the temporal feature extractor , converting raw market data into a dense, state-aware vector that informs subsequent volume allocation adjustments.

2.4.2. Transformer Architectures for Global/Long-Range Dependencies

Transformer models, leveraging their self-attention mechanism, are effective at modeling the characteristic **U-shaped intraday volume pattern** and handling large sequences of historical data by capturing long-term dependencies across the entire trading horizon.

The **U-shape Transformer** proposed by Kim et al. (2023, pp. 2, 6, 17–18, 25–26) specifically focuses on handling day-to-day variations and oscillations in the volume distribution by processing historical volume ratios through a specialized encoder-decoder structure and is based on a **dual-level decomposition strategy** for order distribution. The first level utilizes a **Transformer** model to capture the global distribution of daily volumes (the U-shape), while the second level uses an **LSTM** model to handle the local distribution of orders within smaller time intervals.

Intuition (TUL Level 1 Allocation):

The TUL approach uses the Transformer for **Level 1 (Global)** volume allocation. The loss function explicitly optimizes the predicted volume ratios (u_{pred}) to track the true cumulative VWAP, ensuring the macro allocation aids the micro-level execution.

$$J_{TF} := E_{days} \left[c_1 \frac{1}{L} \sum_{l=0}^{L-1} (u_{true}^l - u_{pred}^l)^2 + c_2 VAA \right]$$

Element	Role in Optimization
$(u_{true}^l - u_{pred}^l)^2$	Minimizes the deviation between the predicted volume ratio for interval l (u_{pred}^l) and the actual realized ratio (u_{true}^l), enforcing accurate U-shape prediction.
VAA (VWAP Approximation Accuracy)	Measures the slippage between the model's achieved price (MP_{day}) and the market VWAP, ensuring the volume allocation contributes to the ultimate execution goal.

Another transformer based intra-day volume forecasting scheme by Lee & Park (2023, pp. 1–2, 4, 13) introduces an **Intraday Volume Estimator (IVE)**, which uses a log-normal transformation to stabilize volume ratios and provides probabilistic forecasting of means and standard deviations. This approach allows traders to anticipate significant intraday volume spikes, which is crucial for navigating market volatility.

How Transformers Improve Probabilistic Volume Forecasting?

Transformers offer several key advantages over traditional volume forecasting methods:

- **Long-Range Dependencies with Attention:** Unlike LSTMs that struggle with very long sequences, Transformers use **self-attention mechanisms** to weigh the importance of different historical time periods. This is crucial for volume forecasting

because market volume patterns can have dependencies spanning **multiple days** (e.g., Monday effects, monthly cycles) and **cross-asset correlations** can be captured simultaneously across multiple securities. Additionally, the model can learn which historical periods are most relevant for current predictions.

- **Probabilistic Output Heads:** Modern Transformer architectures like the Intraday Volume Estimator (IVE) don't just predict point estimates—they output full probability distributions (often using Student's t-distribution). This provides mean forecasts for expected volume (e.g., standard deviation estimates for uncertainty quantification, confidence intervals for risk-aware execution).
- **Spike Detection Through Uncertainty:** High predicted standard deviation correlates strongly with actual volume spikes. This allows the VWAP system to anticipate potential liquidity surges, adjust execution aggressiveness proactively and implement dynamic risk buffers during high-uncertainty periods.
- **Cross-Sectional Learning:** Transformers can process multiple assets simultaneously, learning commonality patterns across sector movements (tech stocks moving together), index rebalancing effects and macro-driven liquidity shifts. This borrowing of information significantly improves prediction accuracy compared to single-asset models.

Comparing TUL and IVE

Model	Technique	Intuition
U-shape Transformer (TUL)	Uses a Transformer Encoder-Decoder architecture specifically to capture day-to-day variations (oscillations) of the U-shaped volume distribution. It predicts the global volume ratio allocation for the entire day.	The Transformer processes historical daily volume ratios (Encoder) and incorporates real-time execution context (Decoder, sometimes using hidden states from an LSTM) to dynamically predict the overall volume profile, moving beyond static historical statistical averages.

Model	Technique	Intuition
IVE (Intraday Volume Estimator)	An encoder-decoder Transformer structure combined with a distribution head (Student's t-distribution) for probabilistic forecasting of volume ratios at a minute-level scale. Inputs include log-transformed ratios, external features, and time encoding.	This model treats volume forecasting as a complex time-series prediction problem, utilizing self-attention for long context (full day) analysis and providing a probabilistic output (mean and standard deviation) to anticipate significant intraday volume spikes.

Applying Transformers in Volume Forecasting

The following scheme is an example showing how we can integrate Transformer-based volume forecasting into a production VWAP system:

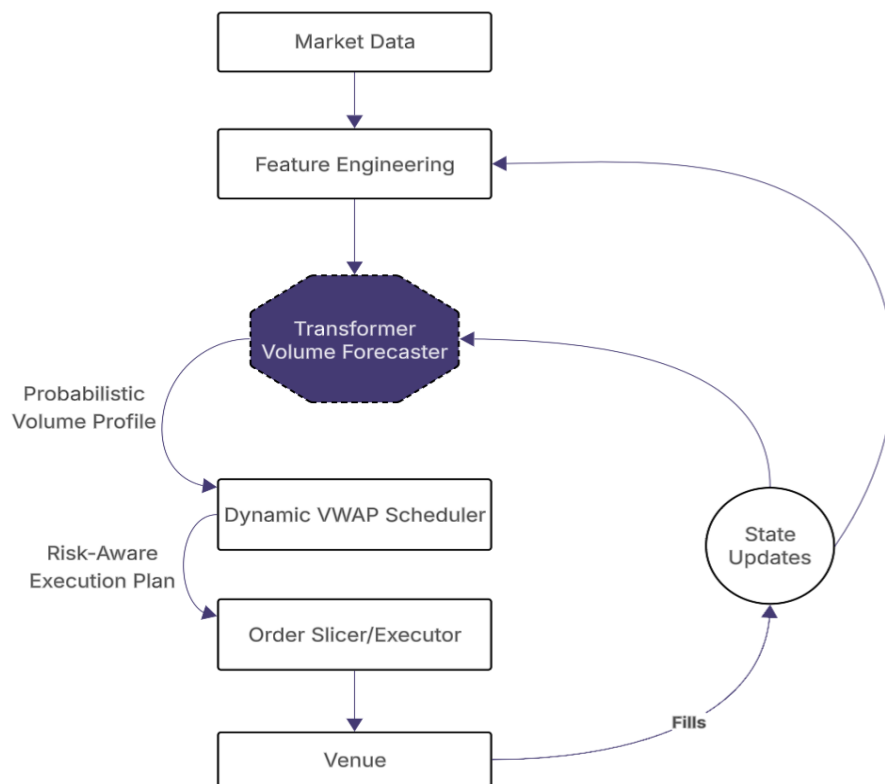


Figure 15: Application of Transformers in Volume Forecasting

1. Feature Engineering Layer: the feature engineering layer comprises classes of features e.g., *input features* (e.g., historical volume ratios, LOB data, time-of-day encoding, sector activity), *temporal features* (e.g., rolling statistics, volatility measures, market regime indicators) and, *cross-sectional features* (e.g., cluster-based volume correlations, index flows).

2. Transformer Forecasting Engine: contains an *encoder* (which processes historical context (past N intervals across M assets)), a *decoder* (to generate future volume distribution for remaining trading day), an *output* (consisting of mean μ_t and standard deviation σ_t for each time bucket) and specification of an *update frequency* (e.g., every 5-15 minutes or on significant market events).

Transformers provide not just better point forecasts, but actionable uncertainty estimates that enable truly adaptive, risk-aware VWAP execution strategies.

2.4.3. Neural Networks for Volume Forecasting (DeepLOB)

Cucuringu et al. (2025, pp. 1–3, 19–20) utilize a **DeepLOB neural network** design enriched with auxiliary high-frequency predictors, such as order flow imbalance, to capture intraday commonalities across stocks. They find that more accurate forecasting models directly result in a reduction of VWAP tracking error and an increased fill ratio for passive orders.

A sample scheme of this design is shown here:

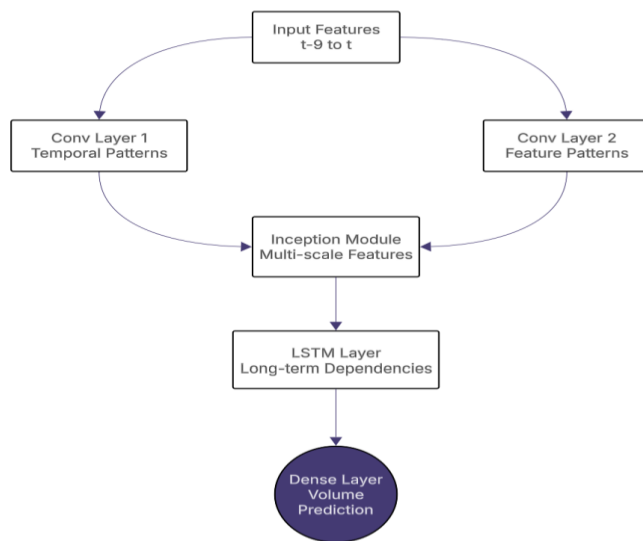


Figure 16: DeepLOB Architecture

This system contains:

1. **Convolutional Layers:** to extract local patterns e.g.,
 - Temporal patterns: “Volume tends to spike after large price moves”
 - Spatial patterns: “Multiple features move together”
2. **Inception Module:** to captures multi-scale relationships e.g.,
 - Short-term: Last 15 minutes behavior
 - Medium-term: Last hour trends
 - Long-term: Multi-day patterns
3. **LSTM:** to remember long-term dependencies e.g.,
 - “Mondays typically have higher volume than Fridays”
 - “Volume increases before major economic announcements”

Comparison of Volume Forecasting Schemes

Accurate forecasts of V_t (or relative volume curve $g_t = V_t/Q_T$) are central to VWAP performance. Below is a compact comparison.

Method	Strengths	Weaknesses
Historical average / U-shape template	Very robust; low data needs	Misses day-specific spikes
Time-series (ARIMA, CMEM)	Captures autocorrelation; interpretable (Brownlees et al.)	Needs careful calibration
State-space / Kalman	Handles missing data; adaptive	Model design required
Machine learning (GBM, RF)	Nonlinear patterns; cross-sectional features	Requires feature engineering
Deep learning (LSTM, Transformer)	Captures long dependencies; high accuracy (IVE paper)	Data hungry; risk of overfitting

In practice a **blended design approach** can be employed for volume forecasting which begins with a **baseline template** (to compute long-run average relative volume curve g_t^{avg} i.e. U-shape), allowing **short-term adjustment** (to fit a lightweight time-series model e.g. AR or

CMEM) on residuals to capture day-specific deviations, incorporates **cross-sectional signals** (e.g., market-wide features - index volume, sector flows). The design comprises a **probabilistic head** (to produce mean and std deviation of V_t in a Transformer or heteroskedastic model to quantify uncertainty and set risk buffers).

The overall system applies an **ensemble** approach combining the template, time-series and ML via weighted blending; using cross-validation on historical days. This way the system instills:

- **probabilistic forecasts** (mean μ_t , std σ_t) in the system design which enables **conservative scheduling** by increasing execution in periods with high predicted volume uncertainty to avoid missing VWAP exposure and,
- applies **dynamic risk limits** to adjust aggressiveness by confidence intervals (e.g., execute to match $\mu_t + \alpha\sigma_t$ when risk-averse).

3. VWAP Schedules (Static, Dynamic, Adaptive)

In the context of **VWAP execution**, a **schedule** (or *trading schedule*) refers to a time-dependent plan that specifies exactly how many units to trade at each moment (or time interval) over the execution horizon — designed to minimize cost, track VWAP, and manage risk.

A schedule represents a **control signal** with **inputs** (e.g., market conditions, risk parameters, volume forecasts) and **output** (v_t - the instantaneous trading rate e.g., 5000 units/minute at 10:15 AM)

*A schedule answers the question **how much should be traded right now — and how will that change over time?***

Formally, it's a function:

$$v_t = f(t; \text{forecasts, risk, inventory})$$

where:

- t in $[0, T]$: current time within trading window (e.g., 9:30 AM – 4:00 PM)
- v_t : asset unit per unit time to execute at time t

- Depends on: volume forecasts (V_t), volatility (σ_t), risk aversion (γ), remaining inventory (q_t), impact costs (k, η)

VWAP execution schedules can be classified as **Static**, **Dynamic**, and **Adaptive** and they aren't redundant — they serve distinct purposes, target different risk profiles, satisfy varying operational constraints, and map to real-world institutional mandates. They are usually deployed as complementary tools in a production execution toolkit.

Białkowski et al. (2008, pp. 670–671, 688, 921–922, 939) provide foundational definitions for the execution schedule types. **Static execution** involves predicting the volume shape once at the beginning of the day and never revising the schedule. The authors define **Dynamic execution** (or theoretical execution) as being based on one-step ahead predictions of the specific volume part. Finally, **Adaptive execution** is characterized by adjusting predictions as the day progresses and market information increases, incorporating all achieved past volume into the remaining strategy.

A sophisticated production setup is typically a rhythmic orchestration of these schedules working in tandem:

- Using **Static** for low-risk, high-compliance orders.
- **Dynamic** to price guaranteed contracts and set internal benchmarks.
- **Adaptive** for discretionary, high-alpha mandates — and measure its performance against Static/Dynamic baselines.

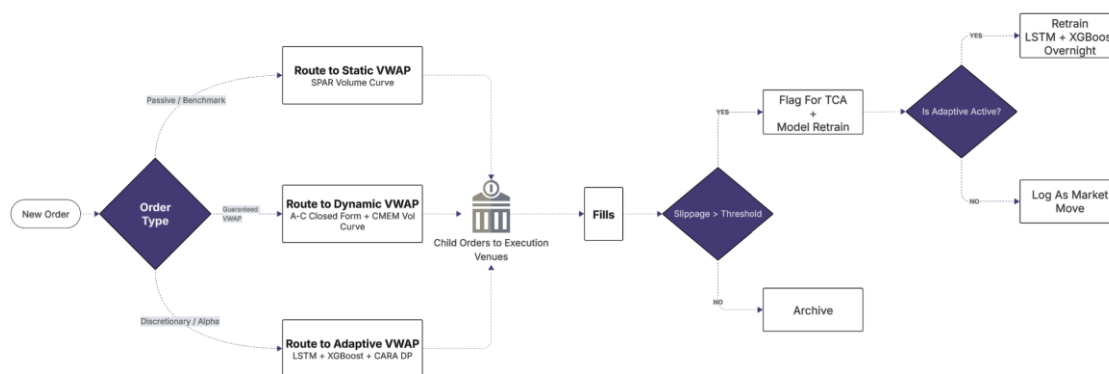


Figure 17: Runtime Routing Among Static, Dynamic and Adaptive Schedules

Having all of them in effect together provides a form of **layered defense** like a car:

- **Static VWAP** = Cruise control (set speed, ignore terrain)

- **Dynamic VWAP** = GPS-guided optimal route (precomputed, assumes no traffic)
- **Adaptive VWAP** = Self-driving AI (reacts to potholes, traffic, weather in real-time)

The system doesn't delete cruise control because it has self-driving. It can use each when it's appropriate. e.g.,

- Dynamic schedule can revert to Static if vol explodes
- Adaptive schedule can fall back to Static if the model drifts

Mechanics of VWAP Schedules

3.1. Static VWAP Schedule

A **static** VWAP schedule follows a precomputed trading curve proportional to expected relative volume:

$$v_t^{\text{static}} = q_0 \cdot \frac{\mathbb{E}[V_t]}{\mathbb{E}[Q_T]}$$

This formula defines a **static VWAP execution schedule**—a simple, widely used strategy for executing a large order in proportion to expected market activity throughout the day.

here,

- v_t^{static} : The planned trading rate (asset units per unit time) at time t .
- q_0 : Total number of asset units the trader needs to execute (“parent order”).
- $\mathbb{E}[V_t]$: Expected market volume *at time t* (e.g., average volume during 10:00–10:01 AM based on historical data).
- $\mathbb{E}[Q_T]$: Expected total market volume over the full trading horizon $[0, T]$, where $Q_T = \int_0^T V_s ds$. So $\mathbb{E}[Q_T] = \int_0^T \mathbb{E}[V_s] ds$.

The strategy says:

“At each moment t , trade a fraction of my total order equal to the fraction of total expected daily volume that occurs at time t .”

In other words:

- If the trader expects **10%** of the day's volume to happen between 10:00–10:15 AM, then they should execute **10% of total order** in that window.
- This ensures the participation rate matches the market's natural liquidity rhythm.

This is a form of **proportional scheduling** or **volume participation**.

Why Is It Called Static?

Because the entire schedule is **computed once at the start of the day** using only *forecasts* ($E[V_t]$), and **never updated**—even if actual volume deviates significantly from expectations.

- It is simple, predictable, easy to implement and low latency.
- But can't adapt to real-time surprises (e.g., a sudden volume spike or drought).

By following this rule, the **personal VWAP** (i.e. the average execution price) should closely track the **market VWAP**, assuming the volume forecast is accurate. This minimizes market impact and satisfies common institutional mandates that require VWAP-based execution.

→ Precomputed curve based on historical average volume profile (e.g., SPAR).

→ “At 10:00 AM, trade 8% of total order because historically 8% of daily volume occurs then.”

A Static VWAP schedule:

- allows the execution of large orders **invisibly**, matching market liquidity rhythm.
- satisfies mandates that require “VWAP benchmark adherence” without active discretion.
- **Regulatory / Client Compliance:** Many passive funds, ETFs, index trackers are contractually obligated to “execute at VWAP” — static VWAP is auditable, explainable, deterministic.
- Does not need real-time optimization, therefore, it's fast, there is no model risk, nor a feedback loop is required.
- **Baseline for TCA:** Brokers and clients use static VWAP as the “dumb baseline” to measure value-add of smarter strategies.

e.g., an ETF rebalancing \$500M across 500 stocks at month-end → uses static VWAP to avoid signaling, minimize tracking error, and pass compliance audits.

A Static VWAP schedule is ineffective when volume deviates from the historical average (earnings day, flash crash, news event).

3.2. Dynamic VWAP Schedule

If future volumes V_t are known deterministically, the optimal schedule (under quadratic costs and CARA utility) often matches the relative volume curve when permanent impact is negligible.

Mitchell et al. (2013, pp. 1547–1548, 1563–1565, 1725–1726, 1741–1743) classify **dynamic algorithms** that revise the volume profile based on information available at each period of time. The authors derive a **closed-form dynamic solution** that remains optimal as a benchmark for minimizing the standard deviation between the personal and market VWAPs.

A Dynamic VWAP Schedule:

- generates a mathematically **optimal schedule** under known forecasts and risk preferences.
- balances permanent impact, temporary cost, and timing risk analytically.
- It is an essential tool to execute **guaranteed VWAP contracts** where a broker assumes risk and must hedge using closed-form solution to price the guarantee.
- It acts as a **theoretical benchmark** to provide “what optimal schedule looks like” under idealized assumptions which is further used to calibrate adaptive systems.
- It provides an **explainable** schedule where every parameter (γ, k, η, σ) has economic meaning and is easy to justify to risk/compliance.

Example: A sell-side desk pricing a “Guaranteed VWAP” for a pension fund → uses AC closed-form to compute hedging cost and optimal execution path.

Dynamic VWAP has a limitation that it assumes perfect knowledge of V_t, σ_t over $[0, T]$ and breaks if reality diverges.

Dynamic VWAP uses Closed-Form, e.g., Almgren-Chriss and is the risk-controlled optimizer schedule.

$$v_t^* = q_0 \cdot \frac{\kappa \cosh(\kappa(T - t))}{\sinh(\kappa T)}, \quad \kappa = \sqrt{\frac{\gamma \sigma^2}{2\eta}}$$

→ Curved, often U-shaped — trades faster early and late if risk-averse.

→ Computed once, assumes perfect knowledge of future vol and volume.

In the Almgren–Chriss quadratic case with linear permanent impact k and temporary cost ηv^2 :

- When $k = 0$ (no permanent impact): optimal $v_t \propto V_t$.
- When $k > 0$: optimal schedule **front-loads** or **back-loads** depending on sign and risk aversion; permanent impact penalizes cumulative trading, so the schedule adjusts to reduce expected slippage variance. In this case the Hamiltonian system yields a closed-form solution (hyperbolic functions).

Trader must **deviate** when $k > 0$:

- **Front-load** if trader is risk-averse and wants to finish early (avoid volatility)
- **Back-load** if trader wants to minimize permanent cost (but risk adverse price moves)

Closed Form Solutions and Hamiltonian Systems

This is the mathematical foundation for optimal VWAP execution strategies under different market impact conditions. The fundamental distinction between two optimal Strategies in VWAP execution based on whether permanent market impact exists:

1. When Permanent Impact = 0 ($k=0$)

- Optimal Strategy: Trade proportionally to market volume

$$(v_t \propto V_t)$$

- Since the trading doesn't permanently move the market price, we must simply follow the natural liquidity pattern
- This is the “pure” VWAP strategy that matches the relative volume curve exactly

2. When Permanent Impact > 0 ($k > 0$)

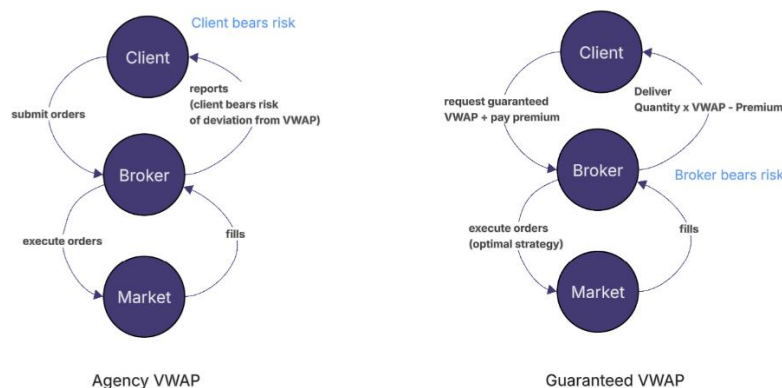
- Optimal Strategy: Deviate from pure proportional scheduling through ‘**front-loading**’ or ‘**backloading**’ because permanent impact means the trades permanently shift the market price against the trader, so the algorithm must balance between:
 - Trading too aggressively (high permanent impact cost)
 - Trading too slowly (high timing risk from price volatility)

This is grounded in the **Almgren-Chriss framework**, which models trading costs as having two components:

- Permanent impact (coefficient k): Linear cost that permanently moves the price
- Temporary impact (coefficient η): Quadratic cost from crossing the spread and eating into order book depth

When both impacts exist, the optimal solution involves hyperbolic functions that represent a sophisticated trade-off between these competing costs.

Implications



- For **agency VWAP** (client bears risk): Stick closer to proportional volume matching since we're not responsible for permanent impact costs.
- For **guaranteed VWAP** (broker bears risk): The broker has incentive to frontload trading to manipulate the VWAP benchmark downward, especially when permanent impact is significant.

This theoretical foundation explains why real-world VWAP algorithms are more sophisticated than simple volume-proportional execution—they must account for how their own trading activity affects market prices.

Significance of Closed Form Solutions and Hamiltonian Systems

The mention of ‘**Closed-form solutions**’ and ‘**Hamiltonian systems**’ in the VWAP context is deeply rooted in the mathematical framework used to solve optimal control problems in trading.

When we say there is a “Closed-form Solution” in existence, that means we can write down the exact optimal trading strategy as an explicit mathematical formula, rather than needing numerical methods or simulations.

e.g.,

- **when $k=0$** (no permanent impact): The closed-form solution is simply $v_t \propto V_t$ (trade proportionally to volume)
- **when $k>0$** (with permanent impact): The solution involves hyperbolic functions that represent the optimal trade-off between impact costs and timing risk

The mention of hyperbolic functions specifically refers to solutions of the **Hamilton-Jacobi-Bellman (HJB) equation**, which is the partial differential equation that characterizes optimal control problems. When we solve the HJB equation for the **Almgren-Chriss** model with both permanent and temporary impact, we get solutions involving hyperbolic sine and cosine functions.

Hamiltonian Systems in VWAP’s Context

The VWAP execution problem is fundamentally a **continuous-time optimal control problem**. We’re trying to find the optimal trading trajectory v_t (our execution speed over time) that minimizes a cost function subject to constraints. In optimal control theory, the **Hamiltonian** is the central mathematical object that combines:

- the objective function (what we want to minimize/maximize)
- the system dynamics (how our state evolves)
- the constraints

For VWAP execution, the Hamiltonian would typically look like:

$$H = Cost_{Term} + \lambda \cdot State_{Dynamics}$$

where:

- **$Cost_{Term}$** includes market impact costs and risk penalties
- λ is the “co-state” variable (like a Lagrange multiplier for dynamic constraints)
- **$State_{Dynamics}$** describes how the remaining inventory evolves: $dq_t/dt = -v_t$

Having closed-form solutions is crucial because the system can calculate the optimal strategy instantly without running complex numerical optimizations during live trading (**computational efficiency**), can understand exactly how parameters affect behavior (i.e. **achieve analytical insights** e.g., “as risk aversion γ increases, we front-load more”), can implement these formulas directly rather than relying on iterative algorithms (**implementation simplicity**) and the numerical implementations can be tested against known analytical solutions (**verifiability**).

3.3. Adaptive/Stochastic VWAP Schedule

When future volume V_t is stochastic, adaptive strategies that update v_t using realized volume and forecasts outperform static schedules. The optimal adaptive control can be derived via **dynamic programming (DP)** (Mitchell et al.; Kato) or approximated using second-order expansions (Kato).

Adaptive VWAP strategy is expressed as a **state-feedback law** by Cheng et al. (2017, pp. 1–3, 9) which adjusts the trading rate in real time based on the investor's current position to ensure the target volume is tracked despite order fill uncertainty. This adaptive approach recovers the classical VWAP strategy when final block trades are strictly prohibited.

It can be visualized as below — the schedule is continuously optimizing the execution path using real-time market data and probabilistic forecasts. It is more complex but is vastly superior in the volatile or unpredictable markets.

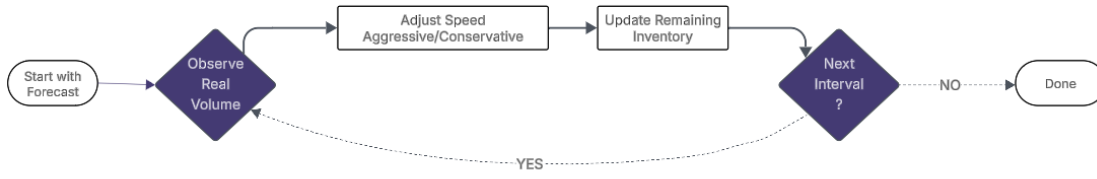


Figure 18: A Typical Adaptive VWAP Workflow

- Reacts to volume surprises, volatility spikes, spread blowouts.
- Aims to maximize P&L in volatile markets e.g., when SPAR fails, this schedule uses LSTM/ML to front-load if volume dries up, backs off if spread explodes.
- It handles uncertainty using CARA utility and stochastic control and doesn't just minimize cost, minimizes the *risk-adjusted* cost.
- This schedule attempts to exploit microstructure knowledge using DeepLOB/Transformer models to detect hidden liquidity and adjust child order size/type in real-time.

Example: Hedge fund executing \$100M Apple block during Fed announcement → adaptive algo slows down when order book thins, accelerates when volume floods in.

A tradeoff of Adaptive schedule is that it incurs higher latency, model drift risk, and is harder to audit and thus is not suitable for regulated/passive mandates. Practically, such algorithms recompute schedule at regular intervals using updated forecasts and remaining inventory.

$$v_t^* = \text{solve} \left(\min \mathbb{E}[\text{cost}] + \frac{\gamma}{2} \text{Var}(\text{cost}) \right) \quad \text{subject to} \quad \int_0^T v_t \, dt = q_0$$

The constraint ensures that the total units traded over the horizon $[0, T]$ equals the initial parent order size q_0 . Without it, the optimization problem is mathematically incomplete.

To complete the formula for the **Adaptive VWAP schedule** based on the mathematical notation and optimal execution frameworks provided, the full expression is as follows:

$$v_t^* = \text{solve} \left(\min \mathbb{E}[\text{cost}] + \frac{\gamma}{2} \text{Var}(\text{cost}) \right) \quad \text{subject to} \quad \int_0^T v_t \, dt = q_0$$

here,

1. **The Objective Function:** It seeks to minimize the risk-adjusted cost of execution.

- $\mathbb{E}[\text{cost}]$: The expected transaction cost, which in the Almgren-Chriss (A-C) model includes **Temporary Impact** (ηv_t^2) and **Permanent Impact** (the price drift $-kv_t$).
 - $\frac{\gamma}{2} \text{Var}(\text{cost})$: The risk penalty. γ is the **CARA (Constant Absolute Risk Aversion)** coefficient. This term penalizes the variance of the slippage (tracking error) relative to the market VWAP.
2. **The Constraint:** $\int_0^T v_t dt = q_0$ (or X_0 in some notations) ensures the total initial position is fully liquidated or acquired by the end of the trading horizon T .
3. **The Adaptive Component:** The “solve” operation is a dynamic process. Unlike a static schedule that is fixed at the start of the day, an **Adaptive VWAP** re-optimizes the trading rate v_t^* at regular intervals (e.g., every 1–5 minutes) using:
- **Real-time Volume Forecasts** (V_t): Updated using models like CMEM, SPAR, or LSTMs.
 - **Remaining Inventory** (q_t): Adjusting for realized fills and market volume surprises.

Connection to the Closed-Form Solution

When the specific quadratic assumptions of the A-C model hold, the “**solution**” to this optimization results in the following closed-form expression:

$$v_t^* = q_0 \cdot \frac{\kappa \cosh(\kappa(T-t))}{\sinh(\kappa T)} \quad \text{where} \quad \kappa = \sqrt{\frac{\gamma \sigma^2}{2\eta}}$$

This produces the mathematically optimal trading curve that balances **market impact** (trading slowly to minimize η) against **timing risk** (trading quickly to minimize σ exposure).

Dynamic Programming (DP) from Adaptive VWAP Perspective

In the stochastic volume case, future market volume V_t at time t is unknown. Only realized volume can be observed as time progresses. This creates a fundamental uncertainty that requires sequential decision-making under uncertainty - exactly what **DP** is designed to solve.

Implications of Dynamic Programming (DP) in VWAP Context

DP breaks down the complex multi-period optimization problem into a sequence of simpler single-period problems. At each time t , it makes the optimal decision given:

- the current state (remaining inventory X_t)
- current information (realized volume so far, current market conditions)
- the optimal value function for future periods

The key insight is defining a **value function** $V(t, X_t, V_t)$ that represents the minimum expected cost-to-go from time t onward, given the current state.

For the stochastic volume case, this becomes:

$$V(t, X_t, V_t) = \min_{v_t} E[\text{Immediate Cost} + V(t + dt, X_{t+dt}, V_{t+dt})]$$

where:

- **Immediate Cost** includes temporary market impact: $\eta * (v_t^2 / V_t)$
- **Future Cost** is captured by the value function $V(t + dt, \dots)$
- **Expectation** accounts for uncertainty in future volume V_{t+dt}

Instead of pre-committing to a static schedule, DP gives an **adaptive policy**:

$$v_t^* = \operatorname{argmin}_{v_t} [\eta * (v_t^2 / V_t) + E[V(t + dt, X_t - v_t * dt, V_{t+dt})]]$$

This means the optimal trading rate at time t depends on:

- **Current liquidity** V_t (trade more when volume is high)
- **Remaining inventory** X_t
- **Time remaining** $(T - t)$
- **Expected future volume dynamics**

4. Handling Volume Uncertainty

DP naturally incorporates the volume forecasting model. If the forecast shows high uncertainty (large variance), the DP solution will be more conservative. If the forecast is confident about high future volume, the algorithm can afford to trade less aggressively.

The following diagram shows how DP transforms VWAP execution from static to adaptive:

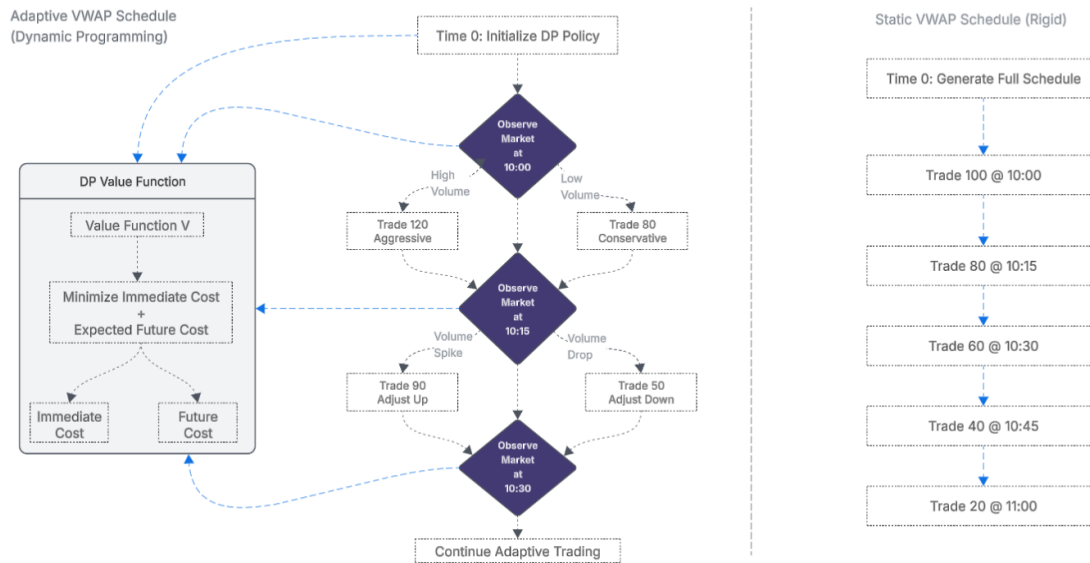


Figure 19: Comparing Adaptive versus Static VWAP Schedules

here,

1. **Static VWAP:** Shows rigid, pre-determined schedule that cannot adapt to market conditions
2. **Adaptive VWAP:** Demonstrates the feedback loop where each decision point observes current market conditions and adjusts accordingly
3. **Value Function:** Represents the mathematical engine that computes optimal decisions by balancing immediate costs against expected future costs
4. **Decision Diamonds:** Show real-time market observation points where the system decides how aggressively to trade based on current liquidity
5. **Arrows to Value Function:** Illustrate how every decision uses the DP value function to make optimal choices

This diagram shows how DP creates an intelligent control system that continuously optimizes trading decisions based on real-time market information, rather than blindly following a pre-computed schedule.

Real-time Adaptation Referring back to **Figure 2**, the closed-loop system continuously:

1. Observes realized volume V_t
2. Updates volume forecasts for remaining periods

3. Recomputes the optimal policy using DP principles
4. Adjusts execution accordingly

Risk Management Integration DP naturally handles the risk-return tradeoff. The value function can include terms for:

- **Timing risk** (variance of tracking error)
- **Market impact costs**
- **Opportunity costs** of incomplete execution

Computational Tractability While the full continuous-time DP leads to PDEs (like the Hamilton-Jacobi-Bellman equation mentioned earlier), in practice we can:

- Discretize time into intervals (e.g., 5-minute buckets)
- Use numerical methods to solve the Bellman equation backward in time
- Pre-compute policy functions for common scenarios
- Update policies in real-time as new information arrives

On the other hand, a **static VWAP** strategy assumes the volume forecast is perfect and never updates. But in reality:

- **Volume forecasts are imperfect** - DP accounts for forecast uncertainty
- **Markets are dynamic** - DP adapts to realized volume deviations
- **Risk preferences matter** - DP incorporates our specific risk tolerance
- **Cost structures are complex** - DP optimally balances multiple cost components

The result is significantly better performance: as noted in the research, dynamic strategies can reduce VWAP tracking error by **37.9%** compared to static approaches, precisely because they use DP principles to adapt optimally to stochastic volume realizations.

DP transforms VWAP execution from a rigid, forecast-dependent schedule into an intelligent, adaptive control system that makes optimal decisions at every moment based on all available information.

Sophisticated systems blend or cascade these VWAP execution schedules forming together a complete, production-grade execution stack., e.g.,

Business Mandate 1 - start with safe, compliant **Static** baseline, overlay with **Dynamic** for risk-controlled deviation (e.g., “if $\text{vol} > 2\sigma$, slow down by 30%”) and, trigger **Adaptive** only when confidence in ML forecast $>$ threshold.

Business Mandate 2 - use **Static** VWAP unless predicted volume error $> 15\%$, then switch to **Adaptive**. If client is Guaranteed VWAP, compute hedge cost via **Dynamic** first.

The rationale for such blending is due to the following observations:

Strategy	Key Idea	Pros	Cons	When to Use
Static VWAP	Follow expected volume curve	Simple, fast, predictable. Good when market behaves predictably, client wants zero discretion, low latency critical.	Can’t adapt to surprises e.g. ineffective if there are volume surprises (earnings, news), spread blows out, volatility regime shift.	Low-risk, liquid markets
Dynamic VWAP	Optimize with known V_t , closed-form	Mathematically optimal under assumptions e.g. trader needs to price a guaranteed contract, explain risk premium to compliance, or benchmark	Requires perfect volume foresight. Performs badly when forecasts are wrong (CMEM/SPAR fails), volatility spikes, liquidity vanishes — no adaptation.	Theoretical or simulated env

Strategy	Key Idea	Pros	Cons	When to Use
Adaptive VWAP	Re-optimize using DP + real data	Robust, responsive, practical. Applicable when markets are chaotic, microstructure matters, trader has quality ML forecasts, and P&L is king.	Computationally heavier and fails when model drifts, latency is too high, overfits noise, and is usually a black-box unacceptable to client.	Real-world, volatile markets

4. Orchestrating Volume Forecasting and Execution Scheduling for Optimal VWAP Performance

Effective VWAP execution hinges on the tight integration of **volume forecasting** (predicting *when* liquidity will appear) and **execution scheduling** (deciding *how fast* to trade). Rather than treating them as isolated modules, pragmatic systems orchestrate them in a **closed-loop** (**forecasts** inform optimal child order allocation via **risk-aware optimizers** - like CARA utility or Almgren-Chriss framework), while real-time fills and market feedback continuously retrain and recalibrate forecasts. **Strategy selection** (Static/Hybrid/Adaptive) is dynamically governed by market regime detection with **hysteresis buffers** to avoid flip-flopping — ensuring robust, low-slippage execution that adapts to volatility, liquidity, and model confidence without human intervention.

The regime detection and subsequent switching is rooted in various studies emphasizing the dynamic selection of execution schedules based on changing market conditions. Białkowski et al. (2008, pp. 9, 20, 25) define **SETAR** (Self-Exciting Threshold Autoregressive) models which explicitly allow for **regime changes** in volume dynamics by applying different parameters when volume exceeds a specific threshold. The authors highlight that the SETAR model's ability to discriminate between turbulent and flat periods is a primary advantage in reducing execution risk.

A **dual-level** system design explored by Kim et al. (2023, pp. 3, 6, 11) blends a **global distribution layer** (Transformer) with a **local distribution layer** (LSTM). The model allows for a choice between a **statistical U-shape method** (based on historical averages) and a **U-shape Transformer method** to capture day-to-day variations that static rules miss. The system is designed to adapt to changing market conditions by incorporating micro-level information (Limit Order Book) to influence macro-level order allocation.

The **Macro-Meta-Micro Trader (M3T)** scheme introduced by Li et al. (2022, pp. 1, 6, 11). incorporates a hierarchical architecture designed to capture market patterns at different **temporal scales**. The **Meta Trader** acts as a bridge, selecting short-term subgoals that are appropriate to the instant liquidity of a specific interval, effectively adjusting the execution speed to match **short-term regime fluctuations**.

Emphasizing the execution schedule optimizer loop in **Figure 3**:

The loop serves as the strategic control hub - determining not only whether to deploy a static, dynamic, or adaptive schedule, but also when to trigger re-optimization, initiate fallback protocols, or refresh (retrain) model parameters.

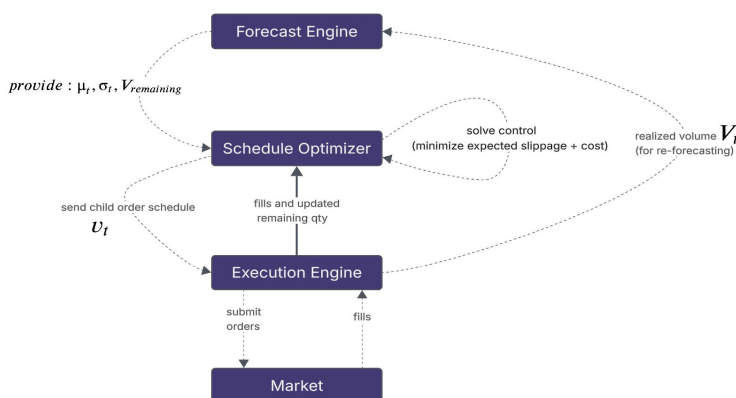


Figure 20: VWAP Schedule Optimization Loop (from Figure 3) Revisited

The following examples illustrate how this orchestration can be achieved in practice.

4.1. Example: Generic Blending

A realistic setup tying **volume forecasting schemes** (CMEM, SPAR, ML e.g. XGBoost, and Deep Learning e.g. LSTM/Transformer) with **execution schedules** (Static, Dynamic, and Adaptive) might look like the following:

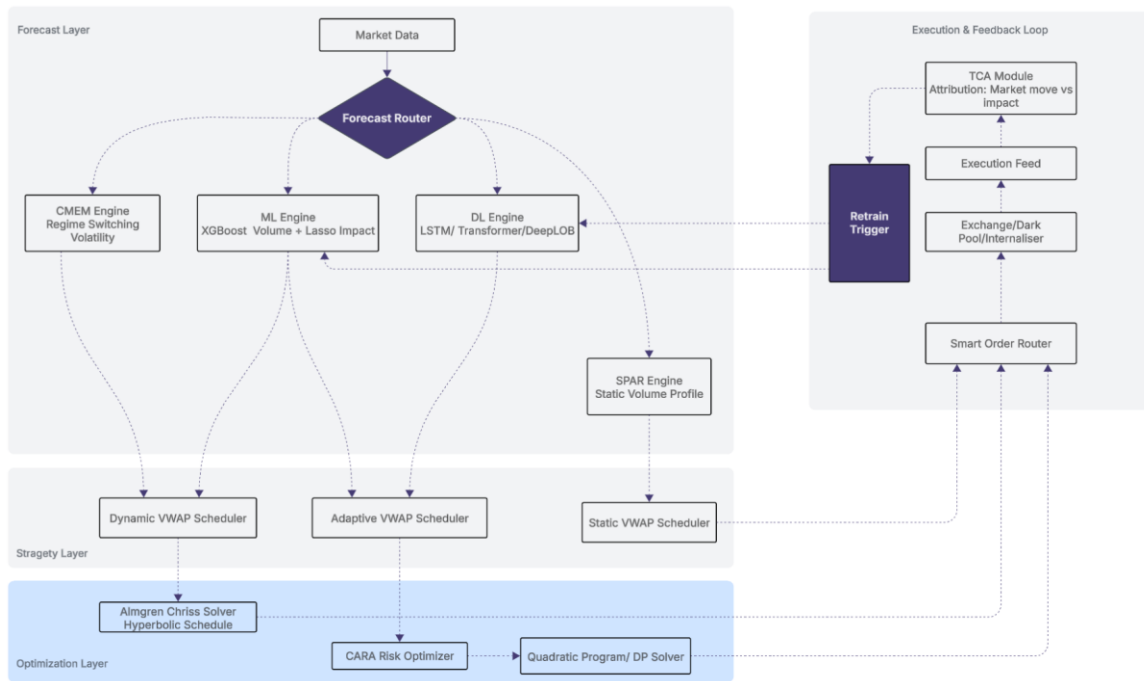


Figure 21: Nexus of VWAP Volume Forecasting and Scheduling Schemes

This system allows deployment of all three strategies in parallel — **static** for compliance and baseline, **dynamic** for pricing, hedging, and theoretical anchoring, **adaptive** for alpha, resilience, and real-world chaos — all fed by the right forecast method, all closing the loop with TCA-driven re-training.

Some observations can be drawn here:

1. SPAR → Static VWAP

- SPAR provides deterministic $\mathbb{E}[V_t]$ curve.
- Scheduler computes $v_t^{\text{static}} = q_0 \cdot \frac{\mathbb{E}[V_t]}{\mathbb{E}[Q_T]}$ once at open.
- No feedback loop — fire and forget.

2. CMEM + Machine Learning (ML) → Dynamic VWAP (Closed-Form)

- CMEM forecasts volatility regimes (σ_t).
- ML (XGBoost) forecasts volume profile (V_t).
- Both assumed **deterministic and known over $[0, T]$** .
- Almgren-Chriss solver generates closed-form hyperbolic schedule.

3. Machine Learning (ML) + Deep Learning (DL) → Adaptive VWAP

- XGBoost: Fast, interpretable volume + spread forecast → updates every 5–15 min.
- LSTM/Transformer: High-frequency microprice or liquidity forecast → adjusts v_t near toxic events.
- CARA optimizer balances expected cost + risk using real-time forecasts.
- Re-solves optimal v_t^* via quadratic program or simplified DP.

4. Feedback Loop: TCA (Transaction Cost Analysis) → Retrain Models

- Slippage attribution (P) identifies:
 - Forecast error (predicted V_t vs. actual)
 - Execution error (impact model mismatch)
- Triggers retraining of:
 - XGBoost volume model (if MAE > threshold)
 - Lasso impact coefficients (if residuals non-stationary)
 - LSTM only on end-of-day (too heavy for intraday retrain)

Deployment And Tuning Considerations

- **Latency Budget:**
 - Static: < 1 ms
 - Dynamic: ~10 ms (precomputed)
 - Adaptive: 50–200 ms (depends on model size)
- **Model Refresh:**
 - SPAR: Daily (exchange update)
 - XGBoost: Hourly or after large slippage
 - LSTM: End-of-day (unless distilled to lightweight proxy)
- **Fallbacks:**
 - If ML/DL fails → fall back to SPAR + static VWAP

- If volatility spikes $\rightarrow$ switch from XGBoost to CMEM for σ_t

Setups like this mirror trading systems — where forecast quality directly shapes execution aggression, and post-trade analytics close the loop for continuous improvement.

e.g., the following diagram illustrates a use-case-specific setup:

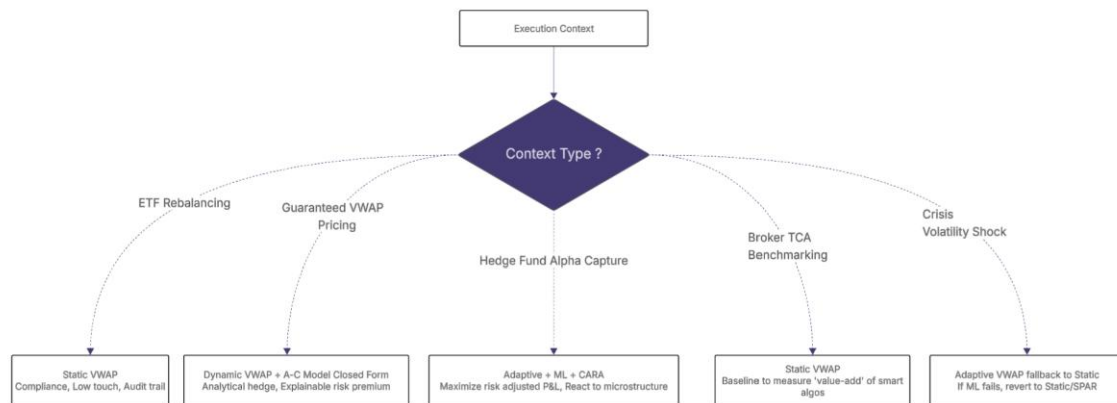


Figure 22: Applying VWAP Schedules and Volume Forecasting to Business Use cases

4.2. Example: Blending Based on Market Regime

Mixing **Static and Adaptive** strategies (or switching between them based on market regime) is a typical production-grade VWAP execution setup. The below diagram shows that:

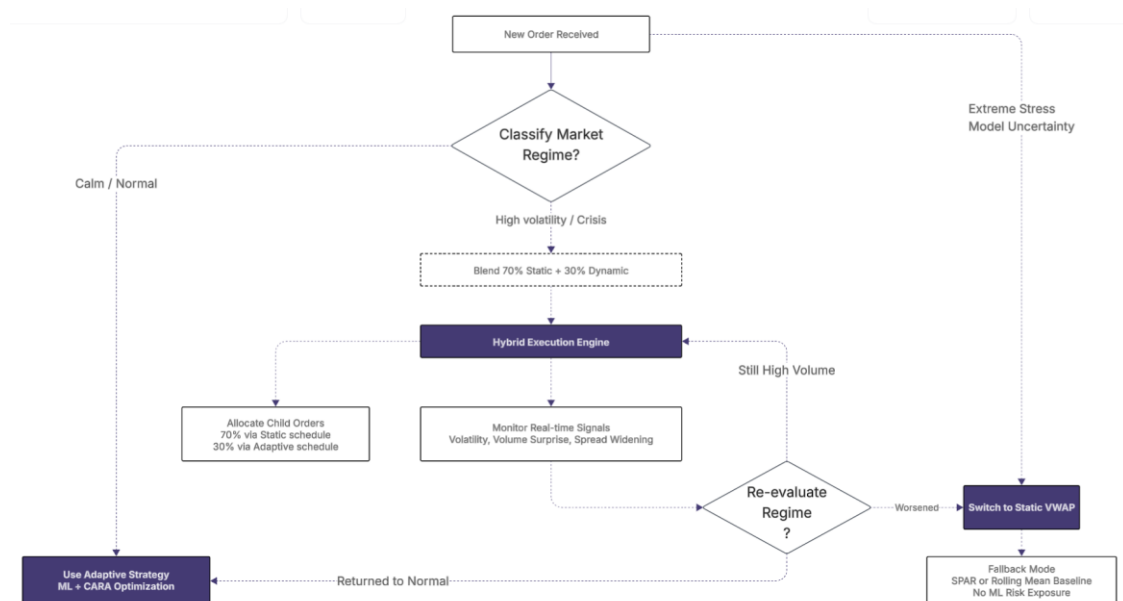


Figure 23: Blending VWAP Schedules Based on Dynamically Changing Market Regimes

Market Regime Detection

Before choosing or blending strategies, classifying the current environment using real-time signals:

- Volatility percentile (e.g., >90th %ile → “High Vol”)
- Volume surprise (forecast error $> 2\sigma$)
- Order book fragility (spread widening, depth collapse)
- ML model confidence score (low → trigger fallback)

e.g., if volatility spikes + volume forecast error exceeds threshold then activate **Hybrid mode**.

Blending Static and Adaptive (Hybrid Strategy)

In high Volatility regimes, instead of going full-static (too rigid) or full-adaptive (too risky), blending can happen:

Example Allocation Logic

if regime == "HighVolatility":

 static_weight = 0.7

 adaptive_weight = 0.3

 schedule = static_weight * static_vwap_curve + adaptive_weight * ml_advisor_signal

Blended profile’s **static** component ensures compliance and avoids wild swings, provides audit trail while the **adaptive** component captures microstructure opportunities (e.g., sudden liquidity pockets). Blending allows a risk-controlled exposure to ML without full dependency.

Fallback to **static** if regime worsens (“Extreme Stress”) or ML model confidence drops below safety threshold:

- Immediately switch to **Static VWAP** (or even simpler: SPAR or Rolling Mean baseline).
- Disable all adaptive logic until regime normalizes.
- Log event for post-trade TCA and model retraining.

Feedback Loop - transition back to **full Adaptive** if in hybrid mode:

- volatility has dropped back to normal
- ML model confidence restored

- the participation rate has exceeded max allowed

Smooth Transitions with Decay Weighting

Instead of hard-switching weights ($0.7 \rightarrow 1.0$), use exponential decay for graceful handoff:

$weight_adaptive = \min(1.0, weight_adaptive_prev + 0.1 * regime_improvement_signal)$

$weight_static = 1.0 - weight_adaptive$

This avoids order schedule discontinuities that could trigger unnecessary market impact.

The following decision-making illustrates the above logic in practice:

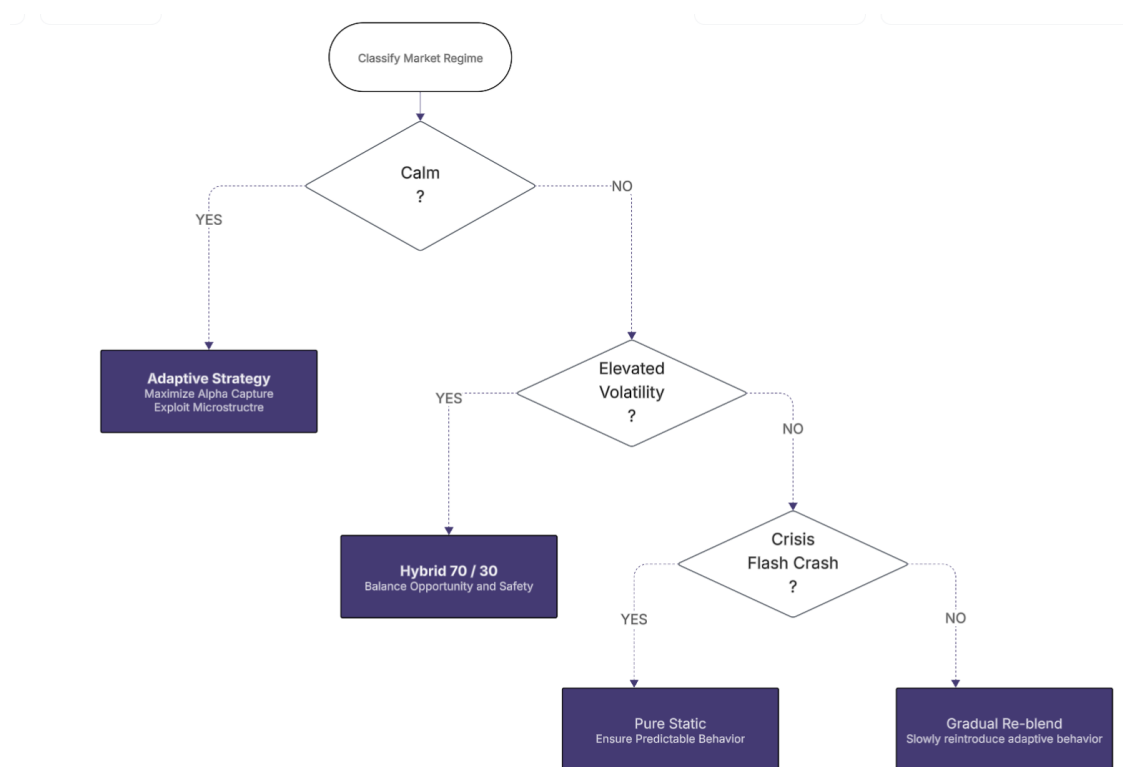


Figure 24: Deciding A VWAP Execution Strategy Based on Market Regime

4.3. Example: Significance of Fallback Thresholds

Setting fallback thresholds for switching between VWAP execution strategies (e.g., from Adaptive $\rightarrow$ Hybrid $\rightarrow$ Static) is critical for risk control, model reliability, and execution robustness — especially under volatile or stressed market conditions.

Below is a framework to define, monitor, and act on fallback thresholds — including **quantitative triggers**, **hysteresis logic**, and **configurable safety layers**.

Core Fallback Thresholds (Quantitative Triggers)

Define real-time metrics with upper/lower bounds that trigger strategy switches:

Metric	Threshold (Example)	Action	Rationale
Realized Volatility	> 95th %ile of 30-day rolling	Switch → Hybrid (70/30)	Avoid overreacting to noise; preserve partial alpha capture
	> 99th %ile OR > 2x median	Switch → Pure Static	Extreme stress → disable ML entirely
Volume Forecast Error	MAE > 2σ of recent forecast errors	Trigger Hybrid	Model losing predictive power
	MAE > 3σ OR sign flip in trend	Trigger Static	Model broken or regime shift
Order Book Fragility	Spread > 5x 10-day median	Trigger Hybrid	Liquidity drying up
	Depth < 10% of 5-day avg	Trigger Static	Market too thin for adaptive slicing
ML Confidence Score	< 0.6 (calibrated probability)	Trigger Hybrid	Model uncertain
	< 0.4 OR NaN returned	Trigger Static	Model failure / crash

Metric	Threshold (Example)	Action	Rationale
Slippage vs Benchmark	Rolling 30-min slippage > -50 bps	Trigger Hybrid	Strategy underperforming
	Slippage > -100 bps OR trending down	Trigger Static	Immediate damage control

Thresholds can be instrument-specific (e.g., AAPL vs HYG) and time-of-day adjusted (e.g., open/close more volatile).

Hysteresis Logic: Avoid “Flip-Flop” Switching

Not switching back immediately when the threshold clears and waiting for confirmed stabilization prevents rapid toggling during choppy markets — e.g., volatility spiking above the threshold, then dipping below, then spiking again within seconds.

Li (2013, pp. 2, 8, 17, 27–28) characterizes the optimal solution for VWAP tracking in terms of a **forward boundary** and a **backward boundary**. These boundaries define an **execution interval** that restricts the total remaining shares to be executed. This mathematical structure establishes three distinct **trading regions: Ahead (A), Behind (B), and Trade (T)**. The **Trade (T) region** effectively serves as a **buffer zone** that tightly tracks the **ideal VWAP trajectory**; the agent places limit orders while within this zone and only executes market orders when the position hits the backward boundary to avoid falling too far "behind" the schedule.

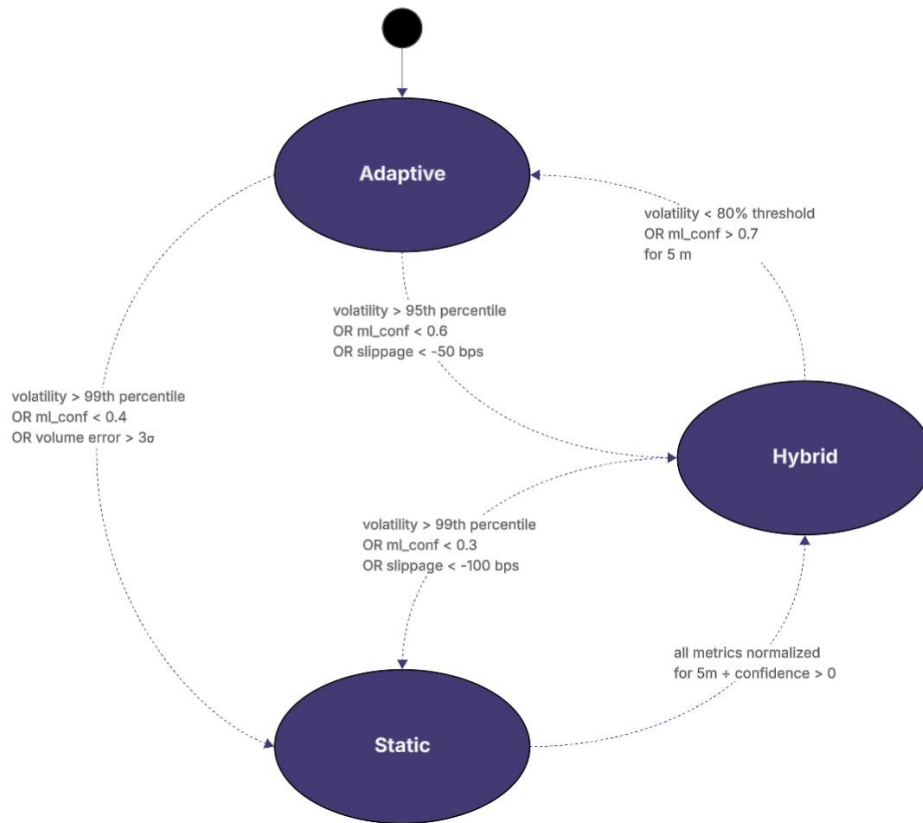


Figure 25: Applying Hysteresis Buffer Zones to Switch Among VWAP Execution Strategies

A similar idea defining **optimal execution boundaries** for the "stationary limit" of VWAP tracking is explored by Cheng et al. (2017, p. 9). They use limit-order and market-order boundaries using the Lambert W function, which creates an operational range where the trader adjusts execution speed based on the current distance from the target trajectory. This **state-feedback expression** functions as a buffer to ensure the target volume is tracked despite the uncertainty of order fills.

Having **hysteresis buffer zones** (intentional delays) prevents chaotic back-and-forth switching by adding “inertia” so the system doesn’t overreact to noise or temporary blips.

Configurable Safety Layers (Runtime Overridable)

Dynamically adjustable configuration turns static fallback logic into an adaptive, data-driven control layer that reduces false triggers, improves alpha capture, and enhances risk management across diverse instruments and sessions.

Allowing overrides via config or risk engine might be useful for pre-market events or known anomalies.

```
strategy_fallback_rules:
  default:
    volatility_95th_percentile: true  # → Hybrid
    volatility_99th_percentile: true  # → Static
    ml_confidence_below: 0.4         # → Static
    max_slippage_bps: -75            # → Hybrid if breached

instruments:
  SPY:
    volatility_multiplier: 1.2       # More tolerant
  HYG:
    volatility_multiplier: 0.8       # More conservative

sessions:
  open_auction:                    # First 30 min
    force_strategy: "Static"
  close_auction:
    allow_adaptive: false          # Disable ML near close
```

Regime classification models like **GMM** (Gaussian Mixture Model), **HMM** (Hidden Markov Model), or simple **Volatility Clustering** automatically detect whether the market is in a “Calm,” “Elevated,” or “Crisis” state by analyzing real-time signals (volatility, spread, volume) — enabling the VWAP execution system to dynamically auto-calibrate strategy thresholds tailored to each asset and time-of-day, rather than relying on brittle, one-size-fits-all rules.

The regime switching can be included in **TCA** (Transaction Cost Analysis) reports: “*Order XYZ switched to Static at 10:14:22 due to volatility spike ($\sigma=4.2$, threshold=3.0) + ML confidence drop (0.32). Slippage saved: -82 bps vs projected -140 bps under Adaptive.*”

A Dynamic Strategy Router in Execution Pipeline

The following is an example of a self-aware risk-adaptive strategy router for VWAP execution systems that can **dynamically route** orders to **Static**, **Hybrid**, or **Adaptive** strategies based on real-time market regime signals.

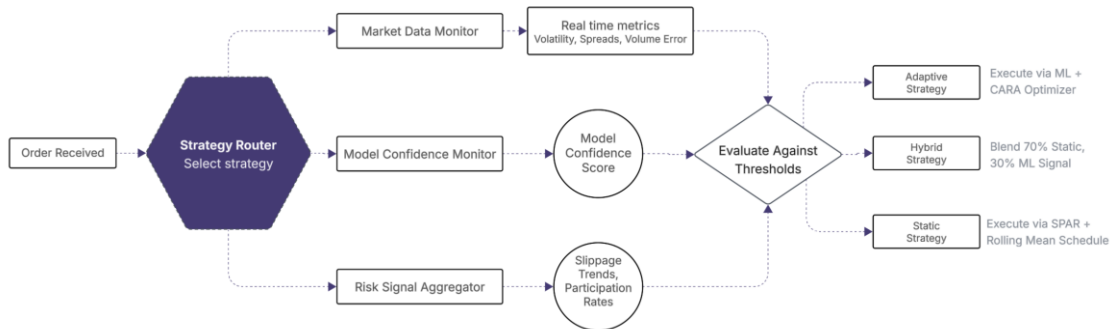


Figure 26: Example Dynamic Strategy Router

This router sits between order ingestion and schedule generation. It consumes real-time metrics and selects the safest and the most effective strategy.

State Machine: Graceful Transitions with Hysteresis Buffer Zones

The router enforces smooth degradation and recovery — avoids chaotic toggling.

Threshold Configuration (Dynamically Loaded):

volatility_95th: 0.025 # 2.5% daily vol

volatility_99th: 0.05 # 5.0%

max_volume_error_sigma: 2.0

min_ml_confidence: 0.6

max_slippage_bps: -50.0

stabilization_minutes: 5

per_instrument_overrides:

HYG:

volatility_95th: 0.04 # More volatile asset → looser trigger

TSLA:

min_ml_confidence: 0.7 # Higher bar for meme stocks

This router design is a template that can turn the execution system into a **self-aware, risk-adaptive engine**.

Optimal VWAP execution isn't about picking the "best" forecast or optimizer — it's about dynamically coupling them, with guardrails, so the system self-adapts to liquidity, volatility, and uncertainty — maximizing execution quality while minimizing catastrophic failure.

Conclusion

This paper has presented a cohesive framework for designing high-performance VWAP execution systems, grounded in the principle that optimal execution emerges not from isolated components, but from the **closed-loop orchestration** of three core pillars: predictive **volume forecasting**, risk-aware **stochastic scheduling**, and adaptive **strategy routing**.

At its foundation, VWAP execution seeks to minimize market impact by aligning order flow with forecasted intraday volume. The accuracy of these forecasts whether derived from classical decomposition (**CMEM, SPAR**), statistical time-series models (**ARMA, Rolling Mean**), or modern machine learning techniques (**Lasso, Transformers**) is critical to reducing tracking error and slippage. Yet forecasting alone is insufficient; execution must be dynamically guided by stochastic control frameworks such as **Almgren–Chriss**, enhanced with **CARA utility** to balance transaction costs, risk, and uncertainty. This transforms probabilistic forecasts into adaptive, risk-calibrated trading schedules.

In practice, however, real-world markets are non-stationary. A **static**, one-size-fits-all execution approach quickly deteriorates under volatility spikes, liquidity shocks, or model drift. Therefore, the paper advocates for an **intelligent strategy router** that continuously classifies market regimes—Calm, Elevated, Crisis—using **clustering, HMM, or GMM** methods, and dynamically selects or blends execution modes (Static, Hybrid, Adaptive). This router employs **hysteresis buffers** to avoid excessive switching and falls back to robust baseline models (e.g., SPAR) when machine learning forecasts degrade, ensuring graceful system resilience.

By marrying forecasting precision, stochastic optimization, and regime-aware adaptation with practical system design, we advance toward a more adaptive, transparent, and robust paradigm for optimal VWAP execution where algorithms do not merely follow the market, but intelligently adapt to the evolving narrative.

References

1. **Almgren, R., & Chriss, N. (2001).** "Optimal execution of portfolio transactions." *Journal of Risk*, 3(2), 5–39
2. **Almgren, R., Thum, C., Hauptmann, E., & Li, H. (2005).** "Direct Estimation of Equity Market Impact." *Risk*, 18(7), 57-62.
3. **Barzykin, A., & Lillo, F. (2019).** "Optimal VWAP execution under transient price impact"
4. **Białkowski, J., Darolles, S., & Le Fol, G. (2008).** "Improving VWAP Strategies: A Dynamic Volume Approach." *Journal of Banking and Finance*, 32(9), 1709-1722.
5. **Brownlees, C. T., Cipollini, F., & Gallo, G. M. (2010).** "Intra-daily Volume Modelling and Prediction for Algorithmic Trading." *Journal of Financial Econometrics*, 9(3), 489-518.
6. **Chen, R., Feng, Y., & Palomar, D. (2016).** "Forecasting Intraday Trading Volume: A Kalman Filter Approach." SSRN Working Paper.
7. **Cheng, X., Di Giacinto, M., & Wang, T.-H. (2017).** "Optimal execution with uncertain order fills in the Almgren-Chriss framework". *Quantitative Finance*, 17(1), 55–69
8. **Cucuringu, M., Li, K., & Zhang, C. (2024).** "Forecasting Intraday Volume in Equity Markets with Machine Learning." (Conference paper, ICLR 2025)
9. **Genet, R. (2025).** "Recurrent neural networks for dynamic VWAP execution: Adaptive trading strategies with temporal Kolmogorov-Arnold networks"
10. **Gu, S., Kelly, B., & Xiu, D. (2020).** "Empirical Asset Pricing via Machine Learning." *Review of Financial Studies*, 33(5), 2223-2273.

11. **Guéant, O., & Royer, G. (2014).** "VWAP Execution and Guaranteed VWAP." SIAM Journal on Financial Mathematics, 5(1), 445-471.
12. **Kato, T. (2017).** "An Optimal Execution Problem with Volume-Dependent Almgren-Chriss Model." SSRN Working Paper.
13. **Kim, S., Kim, J., Sul, H. K., & Hong, Y. (2023).** "An adaptive dual-level reinforcement learning approach for optimal trade execution. Expert Systems with Applications (Preprint)"
14. **Lee, H., & Park, H. (2025).** "IVE: Enhanced Probabilistic Forecasting of Intraday Volume Ratio with Transformers." (Conference paper, ICLR 2025)
15. **Li, X., Wu, P., Zou, C., & Li, Q. (2022).** "Hierarchical deep reinforcement learning for VWAP strategy optimization". Expert Systems with Applications, 194, 116400
16. **Mariotti, T., Lillo, F., & Toscano, G. (2022).** "From zero-intelligence to queue-reactive: Limit order book modelling for high-frequency volatility estimation and optimal execution"
17. **Mitchell, D., Bialkowski, J., & Tompaidis, S. (2013).** "Optimal VWAP Tracking." SSRN Working Paper 2333916.
18. **Skachkov, I. (2013).** "Market impact paradoxes"
19. **Tan, D., Zhang, C., & Zhu, H. (2025).** "Forecasting intraday trading volume with periodicity"